Dokumentasi Resmi & Sintaks Dart (Dart 3.x Up-to-Date)

Bahasa: Dart | Total Chunk/Dokumen: 108 | Disiapkan untuk RAG System

[dart_001] Dart - Variables & Sound Type System

Variables & Sound Null Safety (var, Object, Null safety ?/!, late, final, const): Overview

Sumber: ht

Here's an example of creating a variable and initializing it:
Variables store references. The variable called `name` contains a
reference to a `String` object with a value of "Bob".
The type of the `name` variable is inferred to be `String`,
but you can change that type by specifying it.
If an object isn't restricted to a single type,
specify the `Object` type (or `dynamic` if necessary).
Another option is to explicitly declare the type that would be inferred:
:::note
This page follows the
style guide recommendation
of using `var`, rather than type annotations, for local variables.
:::

Contoh Kode / Perintah CLI:

```
var name = 'Bob';

// ---------------------------------------

Object name = 'Bob';

// ---------------------------------------

String name = 'Bob';
```

[dart_002] Dart - Variables & Sound Type System

Variables & Sound Null Safety (var, Object, Null safety ?/!, late, final, const): Null safety

Sumber: ht

Null safety
The Dart language enforces sound null safety.
Null safety prevents an error that results from unintentional access
of variables set to `null`. The error is called a null dereference error.
A null dereference error occurs when you access a property or call a method
on an expression that evaluates to `null`.
An exception to this rule is when `null` supports the property or method,
like `toString()` or `hashCode`. With null safety, the Dart compiler
detects these potential errors at compile time.
For example, say you want to find the absolute value of an `int` variable `i`.
If `i` is `null`, calling `i.abs()` causes a null dereference error.
In other languages, trying this could lead to a runtime error.
Dart's compiler prevents this error by prohibiting these actions.
Null safety introduces three key changes:
1. When you specify a type for a variable, parameter, or another
relevant component, you can control whether the type allows `null`.

To enable nullability, you add a `?` to the end of the type declaration.
2. You must initialize variables before using them.
Nullable variables default to `null`, so they are initialized by default.
Dart doesn't set initial values to non-nullable types.
It forces you to set an initial value.
Dart doesn't allow you to observe an uninitialized variable.
This prevents you from accessing properties or calling methods
where the receiver's type can be `null`
but `null` doesn't support the method or property used.
3. You can't access properties or call methods on an expression with a
nullable type. The same exception applies where it's a property or method that `null` supports like `hashCode` or `toString()`.
Sound null safety changes potential **runtime errors**
into **edit-time** analysis errors.
Null safety flags a non-null

Contoh Kode / Perintah CLI:

```
String? name  // Nullable type. Can be `null` or string.

    String name   // Non-nullable type. Cannot be `null` but can be string.
```

[dart_003] Dart - Variables & Sound Type System

Variables & Sound Null Safety (var, Object, Null safety ?/!, late, final, const): Default value

Sumber: ht

Default value
Uninitialized variables that have a nullable type
have an initial value of `null`.
Even variables with numeric types are initially null,
because numbers -- like everything else in Dart -- are objects.
:::note
Production code ignores the `assert()` call. During development, on the other
hand, assert(condition) throws an exception if
condition is false. For details, check out [Assert][].
:::
With null safety, you must initialize the values
of non-nullable variables before you use them:
You don't have to initialize a local variable where it's declared,
but you do need to assign it a value before it's used.
For example, the following code is valid because
Dart can detect that `lineCount` is non-null by the time
it's passed to `print()`:
Top-level and class variables are lazily initialized;
the initialization code runs
the first time the variable is used.

Contoh Kode / Perintah CLI:

```
int? lineCount;
assert(lineCount == null);

// ----------------------------------------

int lineCount = 0;
```

```
// ----------------------------------------

int lineCount;

if (weLikeToCount) {
  lineCount = countLines();
} else {
  lineCount = 0;
}

print(lineCount);
```

[dart_004] Dart - Variables & Sound Type System

Variables & Sound Null Safety (var, Object, Null safety ?/!, late, final, const): Late variables

Sumber: ht

Late variables
The `late` modifier has two use cases:
* Declaring a non-nullable variable that's initialized after its declaration.
* Lazily initializing a variable.
Often Dart's control flow analysis can detect when a non-nullable variable
is set to a non-null value before it's used,
but sometimes analysis fails.
Two common cases are top-level variables and instance variables:
Dart often can't determine whether they're set,
so it doesn't try.
If you're sure that a variable is set before it's used,
but Dart disagrees,
you can fix the error by marking the variable as `late`:
:::warning Notice
If you fail to initialize a `late` variable,
a runtime error occurs when the variable is used.
:::
When you mark a variable as `late` but initialize it at its declaration,
then the initializer runs the first time the variable is used.
This lazy initialization is handy in a couple of cases:
* The variable might not be needed,
and initializing it is costly.
* You're initializing an instance variable,
and its initializer needs access to `this`.
In the following example,
if the `temperature` variable is never used,
then the expensive `readThermometer()` function is never called:

Contoh Kode / Perintah CLI:

```
[!late!] String description;

void main() {
  description = 'Feijoada!';
  print(description);
}

// ----------------------------------------

// This is the program's only call to readThermometer().
```

```
[!late!] String temperature = readThermometer(); // Lazily initialized.
```

[dart_005] Dart - Variables & Sound Type System

Variables & Sound Null Safety (var, Object, Null safety ?/!, late, final, const): Final and const

Sumber: ht

Final and const
If you never intend to change a variable, use `final` or `const`, either
instead of `var` or in addition to a type. A final variable can be set
only once; a const variable is a compile-time constant. (Const variables
are implicitly final.)
:::note
[Instance variables][] can be `final` but not `const`.
:::
Here's an example of creating and setting a `final` variable:
You can't change the value of a `final` variable:
Use `const` for variables that you want to be **compile-time constants**. If
the const variable is at the class level, mark it `static const`.
Where you declare the variable, set the value to a compile-time constant
such as a number or string literal, a const
variable, or the result of an arithmetic operation on constant numbers:
The `const` keyword isn't just for declaring constant variables.
You can also use it to create constant _values_,
as well as to declare constructors that _create_ constant values.
Any variable can have a constant value.
You can omit `const` from the initializing expression of a `const` declaration,
like for `baz` above. For details, see [DON'T use const redundantly][].
You can change the reference of a non-final, non-const variable,
even if it used to have a `const` value:
You can't change the value of a `const` variable:
You can define constants that use
[type checks and casts][] (`is` and `as`),
[collection `if`][],
and [spread operators][] (`...` and `...?`):
:::note
Although a `final` object cannot be modified,
its fields can be changed.
In comparison, a `const` object and its fields
cannot be changed: they're _immutable_.
:::
For more information on using `const` to create constant values, see
[Lists][], [Maps][], and [Classes][].

Contoh Kode / Perintah CLI:

```
final name = 'Bob'; // Without a type annotation
final String nickname = 'Bobby';

// ----------------------------------------

Use `const` for variables that you want to be **compile-time constants**. If
the const variable is at the class level, mark it `static const`.
Where you declare the variable, set the value to a compile-time constant
such as a number or string literal, a const
```

```
variable, or the result of an arithmetic operation on constant numbers:

<?code-excerpt "misc/lib/language_tour/variables.dart (const)"?>

// ----------------------------------------

The `const` keyword isn't just for declaring constant variables.
You can also use it to create constant _values_,
as well as to declare constructors that _create_ constant values.
Any variable can have a constant value.

<?code-excerpt "misc/lib/language_tour/variables.dart (const-vs-final)"?>
```

[dart_006] Dart - Variables & Sound Type System

Variables & Sound Null Safety (var, Object, Null safety ?/!, late, final, const): Wildcard variables

Sumber: ht

Wildcard variables
:::version-note
Wildcard variables require
a [language version][] of at least 3.7.
:::
A wildcard variable with the name `_` declares a local variable or parameter
that is non-binding; essentially, a placeholder.
The initializer, if there is one, is still executed, but the value isn't accessible.
Multiple declarations named `_` can exist in the same namespace without a collision error.
Top-level declarations or members where library privacy might be affected are
not valid uses for wildcard variables.
Declarations local to a block scope, such as the following examples,
can declare a wildcard:
* Local variable declaration.
* For loop variable declaration.
* Catch clause parameters.
* Generic type and function type parameters.
* Function parameters.
:::tip
Enable the lint [`unnecessary_underscores`][] to identify where a single
non-binding wildcard variable `_` can replace the previous convention of using
multiple binding underscores (`__`,`___`, etc.) to avoid name collisions.
:::
[Assert]: /language/error-handling#assert
[Instance variables]: /language/classes#instance-variables
[DON'T use const redundantly]: /effective-dart/usage#dont-use-const-redundantly
[type checks and casts]: /language/operators#type-test-operators
[collection `if`]: /language/collections#control-flow-operators
[spread operators]: /language/collections#spread-operators
[Lists]: /language/collections#lists
[Maps]: /language/collections#maps
[Classes]: /language/classes
[language version]: /language/versioning
[`unnecessary_underscores`]: /tools/linter-rules/unnecessary_underscores

Contoh Kode / Perintah CLI:

```
main() {
```

```
    var _ = 1;
    int _ = 2;
  }

// ----------------------------------------

for (var _ in list) {}

// ----------------------------------------

try {
    throw '!';
  } catch (_) {
    print('oops');
  }
```

[dart_007] Dart - Built-in Types & Collections

Built-in Types (Numbers int/double, Strings, Booleans, Runes, Symbols): Overview

Sumber: ht

The Dart language has special support for the following:
- Numbers (`int`, `double`)
- Strings (`String`)
- Booleans (`bool`)
- [Records][] (`(value1, value2)`)
- [Functions][] (`Function`)
- [Lists][] (`List`, also known as *arrays*)
- [Sets][] (`Set`)
- [Maps][] (`Map`)
- Runes (`Runes`; often replaced by the `characters` API)
- Symbols (`Symbol`)
- The value `null` (`Null`)

This support includes the ability to create objects using literals.
For example, `'this is a string'` is a string literal,
and `true` is a boolean literal.
Because every variable in Dart refers to an object -- an instance of a
class -- you can usually use *constructors* to initialize variables. Some
of the built-in types have their own constructors. For example, you can
use the `Map()` constructor to create a map.
Some other types also have special roles in the Dart language:
* `Object`: The superclass of all Dart classes except `Null`.
* `Enum`: The superclass of all enums.
* `Future` and `Stream`: Used in [asynchronous programming][].
* `Iterable`: Used in [for-in loops][iteration] and
in synchronous [generator functions][].
* `Never`: Indicates that an expression can never
successfully finish evaluating.
Most often used for functions that always throw an exception.
* `dynamic`: Indicates that you want to disable static checking.
Usually you should use `Object` or `Object?` instead.
* `void`: Indicates that a value is never used.
Often used as a return type.
The `Object`, `Object?`, `Null`, and `Never` classes
have special roles in the class hierarchy.

Learn about these roles in [Understanding null safety][].
If we decide to cover `dynamic` more,
here's a nice example that illustrates what dynamic does:
dynamic a = 2;
String b = a; // No problem! Until runtime, when you get an uncaught error.
Object

[dart_008] Dart - Built-in Types & Collections

Built-in Types (Numbers int/double, Strings, Booleans, Runes, Symbols): Numbers

Sumber: ht

Numbers
Dart numbers come in two flavors:
[`int`][]
: Integer values no larger than 64 bits,
[depending on the platform][dart-numbers].
On native platforms, values can be from
-263 to 263 - 1.
On the web, integer values are represented as JavaScript numbers
(64-bit floating-point values with no fractional part)
and can be from -253 to 253 - 1.
[`double`][]
: 64-bit (double-precision) floating-point numbers, as specified by
the IEEE 754 standard.
Both `int` and `double` are subtypes of [`num`][].
The num type includes basic operators such as +, -, /, and *,
and is also where you'll find `abs()`,` ceil()`,
and `floor()`, among other methods.
(Bitwise operators, such as \>\>, are defined in the `int` class.)
If num and its subtypes don't have what you're looking for, the
[`dart:math`][] library might.
Integers are numbers without a decimal point. Here are some examples of
defining integer literals:
If a number includes a decimal, it is a double. Here are some examples
of defining double literals:
You can also declare a variable as a num. If you do this, the variable
can have both integer and double values.
Integer literals are automatically converted to doubles when necessary:
Here's how you turn a string into a number, or vice versa:
The `int` type specifies the traditional bitwise shift (`>`, `>>>`),
complement (`~`), AND (`&`), OR (`|`), and XOR (`^`) operators,
which are useful for manipulating and masking flags in bit fields.
For example:
For more examples, see the
[bitwise and shift operator][] section.
Number literals are compile-time constants.
Many arithmetic expressions are also compile-time constants,
as long as their operands are
compile-time constants that evaluate to numbers.
For more information, see [Numbers in Dart][dart-numbers].
You can use one o

Contoh Kode / Perintah CLI:

```
var x = 1;
var hex = 0xDEADBEEF;

// ----------------------------------------

var y = 1.1;
var exponents = 1.42e5;

// ----------------------------------------

num x = 1; // x can have both int and double values
x += 2.5;
```

[dart_009] Dart - Built-in Types & Collections

Built-in Types (Numbers int/double, Strings, Booleans, Runes, Symbols): Strings

Sumber: ht

Strings
A Dart string (`String` object) holds a sequence of UTF-16 code units.
You can use either
single or double quotes to create a string:
You can put the value of an expression inside a string by using
`${`*`expression`*`}`. If the expression is an identifier, you can skip
the `{}`. To get the string corresponding to an object, Dart calls the
object's `toString()` method.
:::note
The `==` operator tests whether two objects are equivalent.
Two strings are equivalent if they contain the
same sequence of code units.
:::
You can concatenate strings using adjacent string literals or the `+`
operator:
To create a multi-line string, use a triple quote with
either single or double quotation marks:
You can create a "raw" string by prefixing it with `r`:
See Runes and grapheme clusters for details on how
to express Unicode characters in a string.
String literals are compile-time constants,
as long as any interpolated expression is a compile-time constant
that evaluates to null or a numeric, string, or boolean value.
For more information on using strings, check out
Strings and regular expressions.

Contoh Kode / Perintah CLI:

```
var s1 = 'Single quotes work well for string literals.';
var s2 = "Double quotes work just as well.";
var s3 = 'It\'s easy to escape the string delimiter.';
var s4 = "It's even easier to use the other delimiter.";

// ----------------------------------------

var s = 'string interpolation';

assert(
  'Dart has $s, which is very handy.' ==
      'Dart has string interpolation, '
          'which is very handy.',
```

```
);
assert(
  'That deserves all caps. '
          '${s.toUpperCase()} is very handy!' ==
      'That deserves all caps. '
          'STRING INTERPOLATION is very handy!',
);

// ---------------------------------------

var s1 =
    'String '
    'concatenation'
    " works even over line breaks.";
assert(
  s1 ==
      'String concatenation works even over '
          'line breaks.',
);

var s2 = 'The + operator ' + 'works, as well.';
assert(s2 == 'The + operator works, as well.');
```

[dart_010] Dart - Built-in Types & Collections

Built-in Types (Numbers int/double, Strings, Booleans, Runes, Symbols): Booleans

Sumber: ht

Booleans
To represent boolean values, Dart has a type named `bool`. Only two
objects have type bool: the boolean literals `true` and `false`,
which are both compile-time constants.
Dart's type safety means that you can't use code like
if (nonBooleanValue) or
assert (nonBooleanValue).
Instead, explicitly check for values, like this:

Contoh Kode / Perintah CLI:

```
// Check for an empty string.
var fullName = '';
assert(fullName.isEmpty);

// Check for zero.
var hitPoints = 0;
assert(hitPoints == 0);

// Check for null.
var unicorn = null;
assert(unicorn == null);

// Check for NaN.
var iMeantToDoThis = 0 / 0;
assert(iMeantToDoThis.isNaN);
```

[dart_011] Dart - Built-in Types & Collections

Built-in Types (Numbers int/double, Strings, Booleans, Runes, Symbols): Runes and grapheme clusters

Sumber: ht

Runes and grapheme clusters
In Dart, [runes][] expose the Unicode code points of a string.
You can use the [characters package][]
to view or manipulate user-perceived characters,
also known as
[Unicode (extended) grapheme clusters.][grapheme clusters]
Unicode defines a unique numeric value for each letter, digit,
and symbol used in all of the world's writing systems.
Because a Dart string is a sequence of UTF-16 code units,
expressing Unicode code points within a string requires
special syntax.
The usual way to express a Unicode code point is
`\uXXXX`, where XXXX is a 4-digit hexadecimal value.
For example, the heart character (?) is `\u2665`.
To specify more or less than 4 hex digits,
place the value in curly brackets.
For example, the laughing emoji (?) is `\u{1f606}`.
If you need to read or write individual Unicode characters,
use the `characters` getter defined on String
by the characters package.
The returned [`Characters`][] object is the string as
a sequence of grapheme clusters.
Here's an example of using the characters API:
The output, depending on your environment, looks something like this:
For details on using the characters package to manipulate strings,
see the [example][characters example] and [API reference][characters API]
for the characters package.

Contoh Kode / Perintah CLI:

```
import 'package:characters/characters.dart';

void main() {
  var hi = 'Hi ??';
  print(hi);
  print('The end of the string: ${hi.substring(hi.length - 1)}');
  print('The last character: ${hi.characters.last}');
}
```

[dart_012] Dart - Built-in Types & Collections

Built-in Types (Numbers int/double, Strings, Booleans, Runes, Symbols): Symbols

Sumber: ht

Symbols
A [`Symbol`][] object
represents an operator or identifier declared in a Dart program. You
might never need to use symbols, but they're invaluable for APIs that
refer to identifiers by name, because minification changes identifier
names but not identifier symbols.
To get the symbol for an identifier, use a symbol literal, which is just

`#` followed by the identifier:
The code from the following excerpt isn't actually what is being shown in the page
Symbol literals are compile-time constants.
[Records]: /language/records
[Functions]: /language/functions#function-types
[Lists]: /language/collections#lists
[Sets]: /language/collections#sets
[Maps]: /language/collections#maps
[asynchronous programming]: /language/async
[iteration]: /libraries/dart-core#iteration
[generator functions]: /language/functions#generators
[Understanding null safety]: /null-safety/understanding-null-safety#top-and-bottom
[`int`]: {{site.dart-api}}/dart-core/int-class.html
[`double`]: {{site.dart-api}}/dart-core/double-class.html
[`num`]: {{site.dart-api}}/dart-core/num-class.html
[`dart:math`]: {{site.dart-api}}/dart-math/dart-math-library.html
[bitwise and shift operator]: /language/operators#bitwise-and-shift-operators
[dart-numbers]: /resources/language/number-representation
[runes]: {{site.dart-api}}/dart-core/Runes-class.html
[characters package]: {{site.pub-pkg}}/characters
[grapheme clusters]: https://unicode.org/reports/tr29/#Grapheme_Cluster_Boundaries
[`Characters`]: {{site.pub-api}}/characters/latest/characters/Characters-class.html
[characters API]: {{site.pub-api}}/characters
[characters example]: {{site.pub-pkg}}/characters/example
[`Symbol`]: {{site.dart-api}}/dart-core/Symbol-class.html
[language version]: /language/versioning

Contoh Kode / Perintah CLI:

```
{% comment %}
The code from the following excerpt isn't actually what is being shown in the page

<?code-excerpt "misc/lib/language_tour/built_in_types.dart (symbols)"?>
```

[dart_013] Dart - Collections & Collection-If/For

Collections (Lists, Sets, Maps, Spread operators ..., Control-flow operators): Overview

Sumber: ht

Dart has built-in support for list, set, and map [collections][].
To learn more about configuring the types collections contain,
check out [Generics][].
[generics]: /language/generics
[collections]: /libraries/dart-core#collections

[dart_014] Dart - Collections & Collection-If/For

Collections (Lists, Sets, Maps, Spread operators ..., Control-flow operators): Lists

Sumber: ht

Lists
Perhaps the most common collection in nearly every programming language
is the *array*, or ordered group of objects. In Dart, arrays are
[`List`][] objects, so most people just call them *lists*.
Dart list literals are denoted by a comma-separated list of
elements enclosed in square brackets (`[]`). Each element
is usually an expression. Here's a simple Dart list:

:::note
Dart infers that `list` has type `List`. If you try to add non-integer
objects to this list, the analyzer or runtime raises an error. For more
information, read about [type inference][].
:::
You can add a comma after the last item in a Dart collection literal.
This _trailing comma_ doesn't affect the collection,
but it can help prevent copy-paste errors.
Lists use zero-based indexing, where 0 is the index of the first element
and `list.length - 1` is the index of the last element.
You can get a list's length using the `.length` property
and access a list's elements using the subscript operator (`[]`):
To create a list that's a compile-time constant,
add `const` before the list literal:
For more information about lists, refer to the Lists section of the
`dart:core` documentation.
[`List`]: {{site.dart-api}}/dart-core/List-class.html
[type inference]: /language/type-system#type-inference

Contoh Kode / Perintah CLI:

```
var list = [1, 2, 3];

// ----------------------------------------

var list = ['Car', 'Boat', 'Plane',];

// ----------------------------------------

var list = [1, 2, 3];
assert(list.length == 3);
assert(list[1] == 2);

list[1] = 1;
assert(list[1] == 1);
```

[dart_015] Dart - Collections & Collection-If/For

Collections (Lists, Sets, Maps, Spread operators ..., Control-flow operators): Sets

Sumber: ht

Sets
A set in Dart is an unordered collection of unique elements.
Dart support for sets is provided by set literals and the
[`Set`][] type.
Here is a simple Dart set, created using a set literal:
:::note
Dart infers that `halogens` has the type `Set`. If you try to add the
wrong type of element to the set, the analyzer or runtime raises an error. For
more information, read about
type inference.
:::
To create an empty set, use `{}` preceded by a type argument,
or assign `{}` to a variable of type `Set`:
:::note Set or map?
The syntax for map literals is similar to that for set
literals. Because map literals came first, `{}` defaults to the `Map` type. If

you forget the type annotation on `{}` or the variable it's assigned to, then
Dart creates an object of type `Map`.
:::
Add items to an existing set using the `add()` or `addAll()` methods:
Use `.length` to get the number of items in the set:
To create a set that's a compile-time constant,
add `const` before the set literal:
For more information about sets, refer to the Sets section of the
`dart:core` documentation.
[`Set`]: {{site.dart-api}}/dart-core/Set-class.html

Contoh Kode / Perintah CLI:

```
var halogens = {'fluorine', 'chlorine', 'bromine', 'iodine', 'astatine'};

// ----------------------------------------

var names = <String>{};
// Set<String> names = {}; // This works, too.
// var names = {}; // Creates a map, not a set.

// ----------------------------------------

var elements = <String>{};
elements.add('fluorine');
elements.addAll(halogens);
```

[dart_016] Dart - Collections & Collection-If/For

Collections (Lists, Sets, Maps, Spread operators ..., Control-flow operators): Maps

Sumber: ht

Maps
In a map, each element is a key-value pair. Each key within
a pair is associated with a value, and both keys and values
can be any type of object. Each key can occur only once,
although the same value can be associated with multiple
different keys. Dart support for maps is provided by
map literals and the [`Map`][] type.
Here are a couple of simple Dart maps, created using map literals:
:::note
Dart infers that `gifts` has the type `Map` and `nobleGases`
has the type `Map`. If you try to add the wrong type of value to
either map, the analyzer or runtime raises an error. For more information,
read about [type inference][].
:::
You can create the same objects using a Map constructor:
:::note
If you come from a language like C# or Java, you might expect to see `new Map()`
instead of just `Map()`. In Dart, the `new` keyword is optional.
For details, see [Using constructors][].
:::
Add a new key-value pair to an existing map
using the subscript assignment operator (`[]=`):
Retrieve a value from a map using the subscript operator (`[]`):
If you look for a key that isn't in a map, you get `null` in return:
Use `.length` to get the number of key-value pairs in the map:

To create a map that's a compile-time constant,
add `const` before the map literal:
For more information about maps, refer to the Maps section of the
`dart:core` documentation.
[Using constructors]: /language/classes#using-constructors
[`Map`]: {{site.dart-api}}/dart-core/Map-class.html
[type inference]: /language/type-system#type-inference

Contoh Kode / Perintah CLI:

```
var gifts = {
  // Key:    Value
  'first': 'partridge',
  'second': 'turtledoves',
  'fifth': 'golden rings',
};

var nobleGases = {2: 'helium', 10: 'neon', 18: 'argon'};

// ----------------------------------------

var gifts = Map<String, String>();
gifts['first'] = 'partridge';
gifts['second'] = 'turtledoves';
gifts['fifth'] = 'golden rings';

var nobleGases = Map<int, String>();
nobleGases[2] = 'helium';
nobleGases[10] = 'neon';
nobleGases[18] = 'argon';

// ----------------------------------------

var gifts = {'first': 'partridge'};
gifts['fourth'] = 'calling birds'; // Add a key-value pair
```

[dart_017] Dart - Collections & Collection-If/For

Collections (Lists, Sets, Maps, Spread operators ..., Control-flow operators): Collection elements {: #collection-elements }

Sumber: ht

Collection elements {: #collection-elements }
A collection literal contains a series of elements. During
runtime, each element is evaluated, producing zero or more
values that are then inserted into the resulting collection.
These elements fall into two main categories: leaf elements
and control flow elements.
* Leaf element: Inserts an individual item into a
collection literal.
* Expression element: Evaluates a single
expression and inserts the resulting value
into the collection.
* Map entry element: Evaluates a pair of key and value
expressions and inserts the resulting entry into the
collection.
* Control flow element: Conditionally or iteratively adds
zero or more values to the surrounding collection.
* Null-aware element: Evaluates an expression and if

the result is not `null`, inserts the value into the surrounding collection.
* Spread element: Iterates over a given sequence (collection expression) and inserts all of the resulting values into the surrounding collection.
* Null-aware spread element: Similar to the spread element, but allows the collection to be `null` and inserts nothing if it is.
* If element: Conditionally evaluates an inner element based on given a condition expression, and optionally evaluates another `else` element if the condition is false.
* For element: Iterates and repeatedly evaluates a given inner element, inserting zero or more resulting values.

To learn more about collection elements, see the following sections.

[dart_018] Dart - Collections & Collection-If/For

Collections (Lists, Sets, Maps, Spread operators ..., Control-flow operators): Expression elements {: #expression-element }

Sumber: ht

Expression elements {: #expression-element }

An expression element evaluates a single expression and inserts the resulting value into a collection. This expression can encompass various constructs like literals, variables, operators, function calls, and constructor calls.

An expression element has this syntax in a collection:

Contoh Kode / Perintah CLI:

```
<expression>
```

[dart_019] Dart - Collections & Collection-If/For

Collections (Lists, Sets, Maps, Spread operators ..., Control-flow operators): Map entry elements {: #map-entry-element }

Sumber: ht

Map entry elements {: #map-entry-element }

A map entry element evaluates a pair of key and value expressions and inserts the resulting entry into a collection. Both the key and the value within this pair can be expressions.

A map entry element has this syntax in a collection:

Contoh Kode / Perintah CLI:

```
<key_expression>: <value_expression>
```

[dart_020] Dart - Collections & Collection-If/For

Collections (Lists, Sets, Maps, Spread operators ..., Control-flow operators): Null-aware elements {: #null-aware-element }

Sumber: ht

Null-aware elements {: #null-aware-element }
A null-aware element evaluates an expression and if the
result is not `null`, inserts the value into the surrounding
collection.
:::version-note
Null-aware collection elements require
a [language version][] of at least 3.8.
:::
A null-aware element has the following syntax in an
expression element:
A null-aware element has the following syntax in a
map entry element:
In the following example, the result for the
null-aware element `?absentValue` is not added to a list called
`items` because `absentValue` is `null`:
The following example illustrates various ways that
you can use null-aware elements inside of
map entry elements:
[language version]: /language/versioning

Contoh Kode / Perintah CLI:

```
?<expression>

// ----------------------------------------

// key is a null-aware element
?<key_expression>: <value_expression>

// ----------------------------------------

// value is a null-aware element
<key_expression>: ?<value_expression>
```

[dart_021] Dart - Collections & Collection-If/For

Collections (Lists, Sets, Maps, Spread operators ..., Control-flow operators): Spread elements {: #spread-element }

Sumber: ht

Spread elements {: #spread-element }
The spread element iterates over a given sequence and
inserts all of the resulting values into the surrounding
collection.
A spread element has the following syntax in a collection.
The sequence expression can represent any expression that
evaluates to an object that implements `Iterable`:
In the following example, the elements in a list called `a`
are added to a list called `items`.
If you are spreading an expression that might evaluate to
`null` and you want to ignore the `null` (and insert no
elements), use a [null-aware spread element][].

To learn more about the spread operator, see
[Spread operator][].
[Spread operator]: /language/operators/#spread-operators
[null-aware spread element]: #null-spread-element

Contoh Kode / Perintah CLI:

```
...<sequence_expression>

// ----------------------------------------

var a = [1, 2, null, 4];
var items = [0, ...a, 5]; // [0, 1, 2, null, 4, 5]
```

[dart_022] Dart - Collections & Collection-If/For

Collections (Lists, Sets, Maps, Spread operators ..., Control-flow operators): Null-aware spread elements {: #null-spread-element }

Sumber: ht

Null-aware spread elements {: #null-spread-element }
The null-aware spread element is similar to the
spread element, but allows the collection to be `null` and
inserts nothing if it is.
A null-aware spread element has this syntax in a
collection:
In the following example, a list called `a` is ignored
because it's null, but the elements in a list called `b`
are added to a list called `items`. Notice that if a
collection itself is not `null`, but it contains elements that
are, those `null` elements are still added to the result.
Because of null safety, you can't perform a
spread operation (`...`) on a value that might be null. The
following example produces a compile-time error because the
`extraOptions` parameter is nullable and the
spread operator used on `extraOptions` is not null-aware.
If you want to spread a nullable collection, use a
null-aware spread element. The following example is valid
because the null-aware spread operator is used on `extraOptions`.
To learn more about the null-aware spread operator, see
[Spread operator][].
[Spread operator]: /language/operators/#spread-operators

Contoh Kode / Perintah CLI:

```
...?<sequence_expression>

// ----------------------------------------

List<int>? a = null;
var b = [1, null, 3];
var items = [0, ...?a, ...?b, 4]; // [0, 1, null, 3, 4]

// ----------------------------------------

If you want to spread a nullable collection, use a
null-aware spread element. The following example is valid
because the null-aware spread operator is used on `extraOptions`.

<?code-excerpt             "misc/test/language_tour/collections/null_spread_operator_in_collection_a.dart
```

```
(code_sample)"?>
```

[dart_023] Dart - Collections & Collection-If/For

Collections (Lists, Sets, Maps, Spread operators ..., Control-flow operators): If elements {: #if-element }

Sumber: ht

If elements {: #if-element }
An `if` element conditionally evaluates an inner element
based on given a condition expression, and optionally
evaluates another `else` element if the condition is false.
The `if` element has a few syntax variations:
The following examples illustrate various ways that
you can use an `if` element inside of a collection with
a boolean expression:
The following examples illustrate various ways that
you can use an `if` element with a `case` part inside of a
collection:
You can mix different `if` operations with an `else if`
part. For example:
To learn more about the `if` conditional, see
[`if` statement][]. To learn more about the `if-case`
conditional, see [`if-case` statement][].
[`if` statement]: /language/branches#if
[`if-case` statement]: /language/branches#if-case

Contoh Kode / Perintah CLI:

```
// If the bool expression is true, include the result.
if (<bool_expression>) <result>

// ----------------------------------------

// If the expression matches the pattern, include the result.
if (<expression> case <pattern>) <result>

// ----------------------------------------

// If the operation resolves as true, include the first
// result, otherwise, include the second result.
if (<bool_expression>) <result> else <result>
```

[dart_024] Dart - Collections & Collection-If/For

Collections (Lists, Sets, Maps, Spread operators ..., Control-flow operators): For elements {: #for-element }

Sumber: ht

For elements {: #for-element }
A `for` element iterates and repeatedly evaluates a given
inner element, inserting zero or more resulting values.
A `for` element has this syntax in a collection:
The following examples illustrate various ways that
you can use a `for` element inside of a collection:
To learn more about the `for` loop, see
[`for` loops][].
[`for` loops]: /language/loops/#for-loops

Contoh Kode / Perintah CLI:

```
for (<expression> in <collection>) <result>

// ----------------------------------------

for (<initialization_clause>; <condition_clause>; <increment_clause>) <result>

// ----------------------------------------

var numbers = [2, 3, 4];
var items = [1, for (var n in numbers) n * n, 7]; // [1, 4, 9, 16, 7]
```

[dart_025] Dart - Collections & Collection-If/For

Collections (Lists, Sets, Maps, Spread operators ..., Control-flow operators): Nest control flow elements {: #nesting-elements }

Sumber: ht

Nest control flow elements {: #nesting-elements }

You can nest control flow elements within each other. This is a powerful alternative to list comprehensions in other languages.

In the following example, only the even numbers in `numbers` are included in `items`.

It's common and idiomatic to use a spread on a collection literal immediately inside of an `if` or `for` element. For example:

You can nest all kinds of elements arbitrarily deep.

In the following example, `if`, `for` and spread elements are nested within each other in a collection:

Contoh Kode / Perintah CLI:

```
var numbers = [1, 2, 3, 4, 5, 6, 7];
var items = [
  0,
  for (var n in numbers)
    if (n.isEven) n,
  8,
]; // [0, 2, 4, 6, 8]

// ----------------------------------------

var items = [
  if (condition) oneThing(),
  if (condition) ...[multiple(), things()],
]; // [oneThing, [multiple_a, multiple_b], things]

// ----------------------------------------

var nestItems = true;
var ys = [1, 2, 3, 4];
var items = [
  if (nestItems) ...[
    for (var x = 0; x < 3; x++)
      for (var y in ys)
        if (x < y) x + y * 10,
  ],
]; // [10, 20, 30, 40, 21, 31, 41, 32, 42]
```

[dart_026] Dart - Functions & First-Class Functions

Functions (Named/Optional parameters, Arrow syntax =>, Anonymous functions, Closures): Overview

Sumber: ht

Dart is a true object-oriented language, so even functions are objects
and have a type, [Function.][Function API reference]
This means that functions can be assigned to variables or passed as arguments
to other functions. You can also call an instance of a Dart class as if
it were a function. For details, see [Callable objects][].
Here's an example of implementing a function:
Although Effective Dart recommends
[type annotations for public APIs][],
the function still works if you omit the types:
For functions that contain just one expression, you can use a shorthand
syntax:
The => expr syntax is a shorthand for
{ return expr; }. The `=>` notation
is sometimes referred to as _arrow_ syntax.
:::note
Only _expressions_ can appear between the arrow (`=>`) and the semicolon (`;`).
Expressions evaluate to values.
This means that you can't write a statement where Dart expects a value.
For example,
you could use a [conditional expression][] but not an [if statement][].
In the previous example,
`_nobleGases[atomicNumber] != null;` returns a boolean value.
The function then returns a boolean value
that indicates whether the `atomicNumber` falls into the noble gas range.
:::

Contoh Kode / Perintah CLI:

```
bool isNoble(int atomicNumber) {
  return _nobleGases[atomicNumber] != null;
}

// ----------------------------------------

isNoble(atomicNumber) {
  return _nobleGases[atomicNumber] != null;
}

// ----------------------------------------

bool isNoble(int atomicNumber) => _nobleGases[atomicNumber] != null;
```

[dart_027] Dart - Functions & First-Class Functions

Functions (Named/Optional parameters, Arrow syntax =>, Anonymous functions, Closures): Parameters

Sumber: ht

Parameters
A function can have any number of _required positional_ parameters. These can be
followed either by _named_ parameters or by _optional positional_ parameters
(but not both).

:::note
Some APIs -- notably [Flutter][] widget constructors -- use only named
parameters, even for parameters that are mandatory. See the next section for
details.
:::
You can use [trailing commas][] when you pass arguments to a function
or when you define function parameters.

[dart_028] Dart - Functions & First-Class Functions

Functions (Named/Optional parameters, Arrow syntax =>, Anonymous functions, Closures): Named parameters

Sumber: ht

Named parameters
Named parameters are optional
unless they're explicitly marked as `required`.
When defining a function, use
{param1, param2, ...}
to specify named parameters.
If you don't provide a default value
or mark a named parameter as `required`,
their types must be nullable
as their default value will be `null`:
When calling a function,
you can specify named arguments using
paramName: value.
For example:
To define a default value for a named parameter besides `null`,
use `=` to specify a default value.
The specified value must be a compile-time constant.
For example:
If you instead want a named parameter to be mandatory,
requiring callers to provide a value for the parameter,
annotate them with `required`:
If someone tries to create a `Scrollbar`
without specifying the `child` argument,
then the analyzer reports an issue.
:::note
A parameter marked as `required` can still be nullable:
:::
You might want to place positional arguments first,
but Dart doesn't require it.
Dart allows named arguments to be placed anywhere in the
argument list when it suits your API:

Contoh Kode / Perintah CLI:

```
/// Sets the [bold] and [hidden] flags ...
void enableFlags({bool? bold, bool? hidden}) {
  ...
}

// ----------------------------------------

enableFlags(bold: true, hidden: false);
```

```
// ----------------------------------------

/// Sets the [bold] and [hidden] flags ...
void enableFlags({bool bold = false, bool hidden = false}) {
  ...
}

// bold will be true; hidden will be false.
enableFlags(bold: true);
```

[dart_029] Dart - Functions & First-Class Functions

Functions (Named/Optional parameters, Arrow syntax =>, Anonymous functions, Closures): Optional positional parameters

Sumber: ht

Optional positional parameters
Wrapping a set of function parameters in `[]`
marks them as optional positional parameters.
If you don't provide a default value,
their types must be nullable
as their default value will be `null`:
Here's an example of calling this function
without the optional parameter:
And here's an example of calling this function with the third parameter:
To define a default value for an optional positional parameter besides `null`,
use `=` to specify a default value.
The specified value must be a compile-time constant.
For example:

Contoh Kode / Perintah CLI:

```
String say(String from, String msg, [String? device]) {
  var result = '$from says $msg';
  if (device != null) {
    result = '$result with a $device';
  }
  return result;
}

// ----------------------------------------

assert(say('Bob', 'Howdy') == 'Bob says Howdy');

// ----------------------------------------

assert(
  say('Bob', 'Howdy', 'smoke signal') ==
      'Bob says Howdy with a smoke signal',
);
```

[dart_030] Dart - Functions & First-Class Functions

Functions (Named/Optional parameters, Arrow syntax =>, Anonymous functions, Closures): Parameter modifiers

Sumber: ht

Parameter modifiers
The `final` and `var` parameter modifiers are reserved exclusively for
[primary constructors][] to declare instance fields.
Using them on formal parameters in any other declaration produces
a compile-time error (`extraneous_modifier`).
For code examples and migration steps, see
[Parameter modifier restrictions][].
To enforce non-reassignable parameters as a style choice,
use the [`parameter_assignments`][] lint rule instead.

[dart_031] Dart - Functions & First-Class Functions

Functions (Named/Optional parameters, Arrow syntax =>, Anonymous functions, Closures): The main() function {:#main}

Sumber: ht

The main() function {:#main}
Every app must have a top-level `main()` function, which serves as the
entrypoint to the app. The `main()` function returns `void` and has an
optional `List` parameter for arguments.
Here's a simple `main()` function:
Here's an example of the `main()` function for a command-line app that
takes arguments:
You can use the args library to
define and parse command-line arguments.

Contoh Kode / Perintah CLI:

```
void main() {
  print('Hello, World!');
}
```

[dart_032] Dart - Functions & First-Class Functions

Functions (Named/Optional parameters, Arrow syntax =>, Anonymous functions, Closures): Functions as first-class objects

Sumber: ht

Functions as first-class objects
You can pass a function as a parameter to another function. For example:
You can also assign a function to a variable, such as:
This example uses an anonymous function.
More about those in the next section.

Contoh Kode / Perintah CLI:

```
void printElement(int element) {
  print(element);
}

var list = [1, 2, 3];

// Pass printElement as a parameter.
list.forEach(printElement);
```

```
// ----------------------------------------

var loudify = (msg) => '!!! ${msg.toUpperCase()} !!!';
assert(loudify('hello') == '!!! HELLO !!!');
```

[dart_033] Dart - Functions & First-Class Functions

Functions (Named/Optional parameters, Arrow syntax =>, Anonymous functions, Closures): Function types

Sumber: ht

Function types
You can specify the type of a function, which is known as a _function type_.
A function type is obtained from a function declaration header by
replacing the function name by the keyword `Function`.
Moreover, you are allowed to omit the names of positional parameters, but
the names of named parameters can't be omitted. For example:
:::note
In Dart, functions are first-class objects,
meaning they can be assigned to variables,
passed as arguments, and returned from other functions.
You can use a [`typedef`][] declaration to explicitly name function types,
which can be useful for clarity and reusability.
:::
[`typedef`]: /language/typedefs

Contoh Kode / Perintah CLI:

```
void greet(String name, {String greeting = 'Hello'}) =>
    print('$greeting $name!');

// Store `greet` in a variable and call it.
void Function(String, {String greeting}) g = greet;
g('Dash', greeting: 'Howdy');
```

[dart_034] Dart - Functions & First-Class Functions

Functions (Named/Optional parameters, Arrow syntax =>, Anonymous functions, Closures): Anonymous functions

Sumber: ht

Anonymous functions
Though you name most functions, such as `main()` or `printElement()`,
you can also create functions without names.
These functions are called _anonymous functions_, _lambdas_, or _closures_.
An anonymous function resembles a named function as it has:
- Zero or more parameters, comma-separated
- Optional type annotations between parentheses.
The following code block contains the function's body:
The following example passes an anonymous function
with an untyped parameter, `item`,
to the `map` function.
The `map` function, invoked for each item in the list,
converts each string to uppercase.
Then, a `for-in` loop prints each converted string with its length.
Click **Run** to execute the code.
If the function contains only a single expression or return statement,

you can shorten it using arrow notation.
Paste the following line into DartPad and click **Run**
to verify that it is functionally equivalent.

Contoh Kode / Perintah CLI:

```
([[Type] param1[, ...]]) {
  codeBlock;
}

// ----------------------------------------

const list = ['apples', 'bananas', 'oranges'];

var uppercaseList = list.map((item) {
  return item.toUpperCase();
}).toList();
// Convert to list after mapping

for (var item in uppercaseList) {
  print('$item: ${item.length}');
}

// ----------------------------------------

If the function contains only a single expression or return statement,
you can shorten it using arrow notation.
Paste the following line into DartPad and click **Run**
to verify that it is functionally equivalent.

<?code-excerpt "misc/test/language_tour/functions_test.dart (anon-func)"?>
```

[dart_035] Dart - Functions & First-Class Functions

Functions (Named/Optional parameters, Arrow syntax =>, Anonymous functions, Closures): Lexical scope

Sumber: ht

Lexical scope
Dart determines the scope of variables based on the layout of its code.
A programming language with this feature is termed a lexically scoped language.
You can "follow the curly braces outwards" to see if a variable is in scope.
Example: A series of nested functions with variables at each scope level:
The `nestedFunction()` method can use variables from every level,
all the way up to the top level.

Contoh Kode / Perintah CLI:

```
bool topLevel = true;

void main() {
  var insideMain = true;

  void myFunction() {
    var insideFunction = true;

    void nestedFunction() {
      var insideNestedFunction = true;

      assert(topLevel);
      assert(insideMain);
      assert(insideFunction);
      assert(insideNestedFunction);
```

```
    }
  }
}
```

[dart_036] Dart - Functions & First-Class Functions

Functions (Named/Optional parameters, Arrow syntax =>, Anonymous functions, Closures): Lexical closures

Sumber: ht

Lexical closures
A function object that can access variables in its lexical scope
when the function sits outside that scope is called a _closure_.
Functions can close over variables defined in surrounding scopes. In the
following example, `makeAdder()` captures the variable `addBy`. Wherever the
returned function goes, it remembers `addBy`.

Contoh Kode / Perintah CLI:

```
/// Returns a function that adds [addBy] to the
/// function's argument.
Function makeAdder(int addBy) {
  return (int i) => addBy + i;
}

void main() {
  // Create a function that adds 2.
  var add2 = makeAdder(2);

  // Create a function that adds 4.
  var add4 = makeAdder(4);

  assert(add2(3) == 5);
  assert(add4(3) == 7);
}
```

[dart_037] Dart - Functions & First-Class Functions

Functions (Named/Optional parameters, Arrow syntax =>, Anonymous functions, Closures): Tear-offs

Sumber: ht

Tear-offs
When you refer to a function, method, or named constructor without parentheses,
Dart creates a _tear-off_. This is a closure that takes the same
parameters as the function and invokes the underlying function when you call it.
If your code needs a closure that invokes a named function with the same
parameters as the closure accepts, don't wrap the call in a lambda.
Use a tear-off.

Contoh Kode / Perintah CLI:

```
var charCodes = [68, 97, 114, 116];
var buffer = StringBuffer();

// ----------------------------------------

<?code-excerpt "misc/lib/language_tour/tear_offs.dart (bad-example)" ?>
```

[dart_038] Dart - Functions & First-Class Functions

Functions (Named/Optional parameters, Arrow syntax =>, Anonymous functions, Closures): Testing functions for equality

Sumber: ht

Testing functions for equality
Here's an example of testing top-level functions, static methods, and
instance methods for equality:

Contoh Kode / Perintah CLI:

```
void foo() {} // A top-level function

class A {
  static void bar() {} // A static method
  void baz() {} // An instance method
}

void main() {
  Function x;

  // Comparing top-level functions.
  x = foo;
  assert(foo == x);

  // Comparing static methods.
  x = A.bar;
  assert(A.bar == x);

  // Comparing instance methods.
  var v = A(); // Instance #1 of A
  var w = A(); // Instance #2 of A
  var y = w;
  x = w.baz;

  // These closures refer to the same instance (#2),
  // so they're equal.
  assert(y.baz == x);

  // These closures refer to different instances,
  // so they're unequal.
  assert(v.baz != w.baz);
}
```

[dart_039] Dart - Functions & First-Class Functions

Functions (Named/Optional parameters, Arrow syntax =>, Anonymous functions, Closures): Return values

Sumber: ht

Return values
All functions return a value. If no return value is specified, the
statement `return null;` is implicitly appended to the function body.
To return multiple values in a function, aggregate the values in a [record][].
[record]: /language/records#multiple-returns

Contoh Kode / Perintah CLI:

```
foo() {}

assert(foo() == null);

// ----------------------------------------
```

```
(String, int) foo() {
  return ('something', 42);
}
```

[dart_040] Dart - Functions & First-Class Functions

Functions (Named/Optional parameters, Arrow syntax =>, Anonymous functions, Closures): Getters and setters

Sumber: ht

Getters and setters
Every property access (top-level, static, or instance) is an invocation
of a getter or a setter. A variable implicitly creates a getter
and, if it's mutable, a setter. This is why when you access a property,
you're actually calling a small function in the background. Reading a
property calls a getter function, and writing one calls a setter function,
even in cases where the property is declared a variable.
However, you can also declare getters and setters explicitly with
the `get` and `set` keywords respectively.
This allows a property's value to be computed when it's read or written.
The purpose of using getters and setters is to create a clear
separation between the client (the code that uses the property) and
the provider (the class or library that defines it). The client asks for or
sets a value without needing to know if that value is stored in a
simple variable or calculated on the spot. This gives the provider
the freedom to change how the property works.
For example, because the value of the property might not be stored anywhere,
it could be computed each time the getter is called. Another example
is that when a value is stored in a private variable, and public access
is only allowed by calling a getter or a setter.
The following example showcases this,
with the `secret` getter and setter providing
indirect access to the private variable `_secret` with
its own manipulations on the assigned and retrieved values.

[dart_041] Dart - Functions & First-Class Functions

Functions (Named/Optional parameters, Arrow syntax =>, Anonymous functions, Closures): Generators

Sumber: ht

Generators
When you need to lazily produce a sequence of values,
consider using a _generator function_.
Dart has built-in support for two kinds of generator functions:
- **Synchronous** generator: Returns an [`Iterable`] object.
- **Asynchronous** generator: Returns a [`Stream`] object.
To implement a **synchronous** generator function,
mark the function body as `sync*`,
and use `yield` statements to deliver values:
To implement an **asynchronous** generator function,
mark the function body as `async*`,
and use `yield` statements to deliver values:
If your generator is recursive,

you can improve its performance by using `yield*`:
[`Iterable`]: {{site.dart-api}}/dart-core/Iterable-class.html
[`Stream`]: {{site.dart-api}}/dart-async/Stream-class.html

Contoh Kode / Perintah CLI:

```
Iterable<int> naturalsTo(int n) sync* {
  int k = 0;
  while (k < n) yield k++;
}

// ----------------------------------------

Stream<int> asynchronousNaturalsTo(int n) async* {
  int k = 0;
  while (k < n) yield k++;
}

// ----------------------------------------

Iterable<int> naturalsDownFrom(int n) sync* {
  if (n > 0) {
    yield n;
    yield* naturalsDownFrom(n - 1);
  }
}
```

[dart_042] Dart - Functions & First-Class Functions

Functions (Named/Optional parameters, Arrow syntax =>, Anonymous functions, Closures): External functions {:#external}

Sumber: ht

External functions {:#external}
An external function is a function whose body is implemented separately from its
declaration. Include the `external` keyword before a function declaration, like so:
An external function's implementation can come from another Dart library,
or, more commonly, from another language. In interop contexts, `external`
introduces type information for foreign functions or values,
making them usable in Dart. Implementation and usage is
heavily platform specific, so check out the interop docs on, for example,
[C][] or [JavaScript][] to learn more.
External functions can be top-level functions, [instance methods][],
getters or setters, or [non-redirecting constructors][].
An [instance variable][] can be `external` too,
which is equivalent to an external getter and (if the variable
is not `final`) an external setter.
[instance methods]: /language/methods#instance-methods
[non-redirecting constructors]: /language/constructors#redirecting-constructors
[instance variable]: /language/classes#instance-variables
[C]: /interop/c-interop
[JavaScript]: /interop/js-interop
[Function API reference]: {{site.dart-api}}/dart-core/Function-class.html
[Callable objects]: /language/callable-objects
[type annotations for public APIs]: /effective-dart/design#do-type-annotate-fields-and-top-level-variables-if-the-type-isnt-obvious
[if statement]: /language/branches#if
[conditional expression]: /language/operators#conditional-expressions

[Flutter]: {{site.flutter}}
[trailing commas]: /language/collections#lists
[primary constructors]: /language/primary-constructors
[`parameter_assignments`]: /tools/linter-rules/parameter_assignments
[Parameter modifier restrictions]: /language/primary-constructors#parameter-modifier-restrictions

Contoh Kode / Perintah CLI:

```
external void someFunc(int i);
```

[dart_043] Dart - Branches & Loops

Branches and Loops (if-case, switch expressions, Exhaustiveness checking): Overview

Sumber: ht

This page shows how you can control the flow of your Dart code using branches:
- `if` statements and elements
- `if-case` statements and elements
- `switch` statements and expressions
You can also manipulate control flow in Dart using:
- [Loops][], like `for` and `while`
- [Exceptions][], like `try`, `catch`, and `throw`

[dart_044] Dart - Branches & Loops

Branches and Loops (if-case, switch expressions, Exhaustiveness checking): If

Sumber: ht

If
Dart supports `if` statements with optional `else` clauses.
The condition in parentheses after `if` must be
an expression that evaluates to a [boolean][]:
To learn how to use `if` in an expression context,
check out [Conditional expressions][].

Contoh Kode / Perintah CLI:

```
if (isRaining()) {
  you.bringRainCoat();
} else if (isSnowing()) {
  you.wearJacket();
} else {
  car.putTopDown();
}
```

[dart_045] Dart - Branches & Loops

Branches and Loops (if-case, switch expressions, Exhaustiveness checking): If-case

Sumber: ht

If-case
Dart `if` statements support `case` clauses followed by a [pattern][]:
If the pattern matches the value,
then the branch executes with any variables the pattern defines in scope.
In the previous example,
the list pattern `[int x, int y]` matches the value `pair`,
so the branch `return Point(x, y)` executes with the variables that
the pattern defined, `x` and `y`.
Otherwise, control flow progresses to the `else` branch
to execute, if there is one:

The if-case statement provides a way to match and
[destructure][] against a _single_ pattern.
To test a value against _multiple_ patterns, use switch.
:::version-note
Case clauses in if statements require
a [language version][] of at least 3.0.
:::

Contoh Kode / Perintah CLI:

```
if (pair case [int x, int y]) return Point(x, y);

// ----------------------------------------

if (pair case [int x, int y]) {
  print('Was coordinate array $x,$y');
} else {
  throw FormatException('Invalid coordinates.');
}
```

[dart_046] Dart - Branches & Loops

Branches and Loops (if-case, switch expressions, Exhaustiveness checking): Switch statements

Sumber: ht

Switch statements
A `switch` statement evaluates a value expression against a series of cases.
Each `case` clause is a [pattern][] for the value to match against.
You can use [any kind of pattern][] for a case.
When the value matches a case's pattern, the case body executes.
Non-empty `case` clauses jump to the end of the switch after completion.
They do not require a `break` statement.
Other valid ways to end a non-empty `case` clause are a
[`continue`][break], [`throw`][], or [`return`][] statement.
Use a `default` or [wildcard `_`][] clause to
execute code when no `case` clause matches:
Empty cases fall through to the next case, allowing cases to share a body.
For an empty case that does not fall through,
use [`break`][break] for its body.
For non-sequential fall-through,
you can use a [`continue` statement][break] and a label:
You can use [logical-or patterns][] to allow cases to share a body or a guard.
To learn more about patterns and case clauses,
check out the patterns documentation on [Switch statements and expressions][].
[Switch statements and expressions]: /language/patterns#switch-statements-and-expressions

Contoh Kode / Perintah CLI:

```
var command = 'OPEN';
switch (command) {
  case 'CLOSED':
    executeClosed();
  case 'PENDING':
    executePending();
  case 'APPROVED':
    executeApproved();
  case 'DENIED':
    executeDenied();
  case 'OPEN':
```

```
    executeOpen();
  default:
    executeUnknown();
}

// ----------------------------------------

switch (command) {
  case 'OPEN':
    executeOpen();
    continue newCase; // Continues executing at the newCase label.

  case 'DENIED': // Empty case falls through.
  case 'CLOSED':
    executeClosed(); // Runs for both DENIED and CLOSED,

  newCase:
  case 'PENDING':
    executeNowClosed(); // Runs for both OPEN and PENDING.
}
```

[dart_047] Dart - Branches & Loops

Branches and Loops (if-case, switch expressions, Exhaustiveness checking): Switch expressions

Sumber: ht

Switch expressions
A `switch` _expression_ produces a value based on the expression
body of whichever case matches.
You can use a switch expression wherever Dart allows expressions,
except at the start of an expression statement. For example:
If you want to use a switch at the start of an expression statement,
use a switch statement.
Switch expressions allow you to rewrite a switch _statement_ like this:
Into an _expression_, like this:
The syntax of a `switch` expression differs from `switch` statement syntax:
- Cases _do not_ start with the `case` keyword.
- A case body is a single expression instead of a series of statements.
- Each case must have a body; there is no implicit fallthrough for empty cases.
- Case patterns are separated from their bodies using `=>` instead of `:`.
- Cases are separated by `,` (and an optional trailing `,` is allowed).
- Default cases can _only_ use `_`, instead of allowing both `default` and `_`.
:::version-note
Switch expressions require a [language version][] of at least 3.0.
:::

Contoh Kode / Perintah CLI:

```
var x = switch (y) { ... };

print(switch (x) { ... });

return switch (x) { ... };

// ----------------------------------------

// Where slash, star, comma, semicolon, etc., are constant variables...
switch (charCode) {
```

```
  case slash || star || plus || minus: // Logical-or pattern
    token = operator(charCode);
  case comma || semicolon: // Logical-or pattern
    token = punctuation(charCode);
  case >= digit0 && <= digit9: // Relational and logical-and patterns
    token = number();
  default:
    throw FormatException('Invalid');
}

// ----------------------------------------

token = switch (charCode) {
  slash || star || plus || minus => operator(charCode),
  comma || semicolon => punctuation(charCode),
  >= digit0 && <= digit9 => number(),
  _ => throw FormatException('Invalid'),
};
```

[dart_048] Dart - Branches & Loops

Branches and Loops (if-case, switch expressions, Exhaustiveness checking): Exhaustiveness checking

Sumber: ht

Exhaustiveness checking
Exhaustiveness checking is a feature that reports a
compile-time error if it's possible for a value to enter a switch but
not match any of the cases.
A default case (`default` or `_`) covers all possible values that
can flow through a switch.
This makes a switch on any type exhaustive.
[Enums][enum] and [sealed types][sealed] are particularly useful for
switches because, even without a default case,
their possible values are known and fully enumerable.
Use the [`sealed` modifier][sealed] on a class to enable
exhaustiveness checking when switching over subtypes of that class:
If anyone were to add a new subclass of `Shape`,
this `switch` expression would be incomplete.
Exhaustiveness checking would inform you of the missing subtype.
This allows you to use Dart in a somewhat
functional algebraic datatype style.

Contoh Kode / Perintah CLI:

```
// Non-exhaustive switch on bool?, missing case to match null possibility:
switch (nullableBool) {
  case true:
    print('yes');
  case false:
    print('no');
}

// ----------------------------------------

sealed class Shape {}

class Square implements Shape {
  final double length;
  Square(this.length);
```

```
}

class Circle implements Shape {
  final double radius;
  Circle(this.radius);
}

double calculateArea(Shape shape) => switch (shape) {
  Square(length: var l) => l * l,
  Circle(radius: var r) => math.pi * r * r,
};
```

[dart_049] Dart - Branches & Loops

Branches and Loops (if-case, switch expressions, Exhaustiveness checking): Guard clause

Sumber: ht

Guard clause
To set an optional guard clause after a `case` clause, use the keyword `when`.
A guard clause can follow `if case`, and
both `switch` statements and expressions.
Guards evaluate an arbitrary boolean expression _after_ matching.
This allows you to add further constraints on
whether a case body should execute.
When the guard clause evaluates to false,
execution proceeds to the next case rather
than exiting the entire switch.
[language version]: /language/versioning
[loops]: /language/loops
[exceptions]: /language/error-handling
[conditional expressions]: /language/operators#conditional-expressions
[boolean]: /language/built-in-types#booleans
[pattern]: /language/patterns
[enum]: /language/enums
[`throw`]: /language/error-handling#throw
[`return`]: /language/functions#return-values
[wildcard `_`]: /language/pattern-types#wildcard
[break]: /language/loops#break-and-continue
[sealed]: /language/class-modifiers#sealed
[any kind of pattern]: /language/pattern-types
[destructure]: /language/patterns#destructuring
[section on switch]: /language/patterns#switch-statements-and-expressions
[logical-or patterns]: /language/patterns#or-pattern-switch

Contoh Kode / Perintah CLI:

```
// Switch statement:
switch (something) {
  case somePattern when some || boolean || expression:
    //             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ Guard clause.
    body;
}

// Switch expression:
var value = switch (something) {
  somePattern when some || boolean || expression => body,
  //              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ Guard clause.
```

```
}

// If-case statement:
if (something case somePattern when some || boolean || expression) {
  //                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ Guard clause.
  body;
}
```

[dart_050] Dart - Classes, Constructors & Mixins

Classes and Objects (Named constructors, Initializer lists, Factory constructors): Overview

Sumber: ht

Dart is an object-oriented language with classes and mixin-based
inheritance. Every object is an instance of a class, and all classes
except `Null` descend from [`Object`][].
Mixin-based inheritance means that although every class
(except for the [top class][top-and-bottom], `Object?`)
has exactly one superclass, a class body can be reused in
multiple class hierarchies.
[Extension methods][] are a way to
add functionality to a class without changing the class or creating a subclass.
[Class modifiers][] allow you to control how libraries can subtype a class.

[dart_051] Dart - Classes, Constructors & Mixins

Classes and Objects (Named constructors, Initializer lists, Factory constructors): Using class members

Sumber: ht

Using class members
Objects have *members* consisting of functions and data (*methods* and
instance variables, respectively). When you call a method, you *invoke*
it on an object: the method has access to that object's functions and
data.
Use a dot (`.`) to refer to an instance variable or method:
Use `?.` instead of `.` to avoid an exception
when the leftmost operand is null:

Contoh Kode / Perintah CLI:

```
var p = Point(2, 2);

// Get the value of y.
assert(p.y == 2);

// Invoke distanceTo() on p.
double distance = p.distanceTo(Point(4, 4));

// ----------------------------------------

// If p is non-null, set a variable equal to its y value.
var a = p?.y;
```

[dart_052] Dart - Classes, Constructors & Mixins

Classes and Objects (Named constructors, Initializer lists, Factory constructors): Using constructors

Sumber: ht

Using constructors
You can create an object using a *constructor*.
Constructor names can be either ClassName or
ClassName.identifier. For example,
the following code creates `Point` objects using the
`Point()` and `Point.fromJson()` constructors:
The following code has the same effect, but
uses the optional `new` keyword before the constructor name:
Some classes provide [constant constructors][].
To create a compile-time constant using a constant constructor,
put the `const` keyword before the constructor name:
Constructing two identical compile-time constants results in a single,
canonical instance:
Within a _constant context_, you can omit the `const` before a constructor
or literal. For example, look at this code, which creates a const map:
You can omit all but the first use of the `const` keyword:
If a constant constructor is outside of a constant context
and is invoked without `const`,
it creates a **non-constant object**:

Contoh Kode / Perintah CLI:

```
var p1 = Point(2, 2);
var p2 = Point.fromJson({'x': 1, 'y': 2});

// ----------------------------------------

var p1 = new Point(2, 2);
var p2 = new Point.fromJson({'x': 1, 'y': 2});

// ----------------------------------------

var p = const ImmutablePoint(2, 2);
```

[dart_053] Dart - Classes, Constructors & Mixins

Classes and Objects (Named constructors, Initializer lists, Factory constructors): Getting an object's type

Sumber: ht

Getting an object's type
To get an object's type at runtime,
you can use the `Object` property `runtimeType`,
which returns a [`Type`][] object.
:::warning
Use a [type test operator][] rather than `runtimeType`
to test an object's type.
In production environments, the test `object is Type` is more stable
than the test `object.runtimeType == Type`.
:::
Up to here, you've seen how to _use_ classes.
The rest of this section shows how to _implement_ classes.

Contoh Kode / Perintah CLI:

```
print('The type of a is ${a.runtimeType}');
```

[dart_054] Dart - Classes, Constructors & Mixins

Classes and Objects (Named constructors, Initializer lists, Factory constructors): Instance variables

Sumber: ht

Instance variables
Here's how you declare instance variables:
An uninitialized instance variable declared with a
[nullable type][] has the value `null`.
Non-nullable instance variables [must be initialized][] at declaration.
All instance variables generate an implicit *getter* method.
Non-final instance variables and
`late final` instance variables without initializers also generate
an implicit *setter* method. For details,
check out [Getters and setters][].
Initializing a non-`late` instance variable where it's declared
sets the value when the instance is created,
before the constructor and its initializer list execute.
As a result, the initializing expression (after the `=`)
of a non-`late` instance variable can't access `this`.
Instance variables can be `final`,
in which case they must be set exactly once.
Initialize `final`, non-`late` instance variables
at declaration,
using a constructor parameter, or
using a constructor's [initializer list][]:
If you need to assign the value of a `final` instance variable
after the constructor body starts, you can use one of the following:
* Compute the value in a [factory constructor][],
then pass it to a generative constructor that
initializes the `final` instance variable.
* Use `late final`, but [_be careful:_][late-final-ivar]
a `late final` without an initializer adds a setter to the API.

Contoh Kode / Perintah CLI:

```
class Point {
  double? x; // Declare instance variable x, initially null.
  double? y; // Declare y, initially null.
  double z = 0; // Declare z, initially 0.
}

// ----------------------------------------

class Point {
  double? x; // Declare instance variable x, initially null.
  double? y; // Declare y, initially null.
}

void main() {
  var point = Point();
  point.x = 4; // Use the setter method for x.
  assert(point.x == 4); // Use the getter method for x.
  assert(point.y == null); // Values default to null.
```

```
}

// ----------------------------------------

double initialX = 1.5;

class Point {
  // OK, can access declarations that do not depend on `this`:
  double? x = initialX;

  // ERROR, can't access `this` in non-`late` initializer:
  double? y = this.x;

  // OK, can access `this` in `late` initializer:
  late double? z = this.x;

  // OK, `this.x` and `this.y` are parameter declarations, not expressions:
  Point(this.x, this.y);
}
```

[dart_055] Dart - Classes, Constructors & Mixins

Classes and Objects (Named constructors, Initializer lists, Factory constructors): Implicit interfaces

Sumber: ht

Implicit interfaces
Every class implicitly defines an interface containing all the instance members of the class and of any interfaces it implements. If you want to create a class A that supports class B's API without inheriting B's implementation, class A should implement the B interface.
A class implements one or more interfaces by declaring them in an `implements` clause and then providing the APIs required by the interfaces. For example:
Here's an example of specifying that a class implements multiple interfaces:

Contoh Kode / Perintah CLI:

```
// A person. The implicit interface contains greet().
class Person {
  // In the interface, but visible only in this library.
  final String _name;

  // Not in the interface, since this is a constructor.
  Person(this._name);

  // In the interface.
  String greet(String who) => 'Hello, $who. I am $_name.';
}

// An implementation of the Person interface.
class Impostor implements Person {
  String get _name => '';

  String greet(String who) => 'Hi $who. Do you know who I am?';
}

String greetBob(Person person) => person.greet('Bob');

void main() {
```

```
  print(greetBob(Person('Kathy')));
  print(greetBob(Impostor()));
}

// ----------------------------------------

class Point implements Comparable, Location {
  ...
}
```

[dart_056] Dart - Classes, Constructors & Mixins

Classes and Objects (Named constructors, Initializer lists, Factory constructors): Class variables and methods

Sumber: ht

Class variables and methods
Use the `static` keyword to implement class-wide variables and methods.

[dart_057] Dart - Classes, Constructors & Mixins

Classes and Objects (Named constructors, Initializer lists, Factory constructors): Static variables

Sumber: ht

Static variables
Static variables (class variables) are useful for class-wide state and
constants:
Static variables aren't initialized until they're used.
:::note
This page follows the
style guide recommendation
of preferring `lowerCamelCase` for constant names.
:::

Contoh Kode / Perintah CLI:

```
class Queue {
  static const initialCapacity = 16;
  // ***
}

void main() {
  assert(Queue.initialCapacity == 16);
}
```

[dart_058] Dart - Classes, Constructors & Mixins

Classes and Objects (Named constructors, Initializer lists, Factory constructors): Static methods

Sumber: ht

Static methods
Static methods (class methods) don't operate on an instance, and thus
don't have access to `this`.
They do, however, have access to static variables.
As the following example shows,
you invoke static methods directly on a class:
:::note

Consider using top-level functions, instead of static methods, for
common or widely used utilities and functionality.
:::
You can use static methods as compile-time constants. For example, you
can pass a static method as a parameter to a constant constructor.
[`Object`]: {{site.dart-api}}/dart-core/Object-class.html
[top-and-bottom]: /null-safety/understanding-null-safety#top-and-bottom
[Extension methods]: /language/extension-methods
[Class modifiers]: /language/class-modifiers
[constant constructors]: /language/constructors#constant-constructors
[`Type`]: {{site.dart-api}}/dart-core/Type-class.html
[type test operator]: /language/operators#type-test-operators
[Getters and setters]: /language/methods#getters-and-setters
[initializer list]: /language/constructors#use-an-initializer-list
[factory constructor]: /language/constructors#factory-constructors
[late-final-ivar]: /effective-dart/design#avoid-public-late-final-fields-without-initializers
[nullable type]: /null-safety/understanding-null-safety#using-nullable-types
[must be initialized]: /null-safety/understanding-null-safety#uninitialized-variables

Contoh Kode / Perintah CLI:

```
import 'dart:math';

class Point {
  double x, y;
  Point(this.x, this.y);

  static double distanceBetween(Point a, Point b) {
    var dx = a.x - b.x;
    var dy = a.y - b.y;
    return sqrt(dx * dx + dy * dy);
  }
}

void main() {
  var a = Point(2, 2);
  var b = Point(4, 4);
  var distance = Point.distanceBetween(a, b);
  assert(2.8 < distance && distance < 2.9);
  print(distance);
}
```

[dart_059] Dart - Modern OOP - Mixins

Mixins (Code reuse across multiple class hierarchies with on & mixin class): Overview

Sumber: ht

Mixins are a way of defining code that can be reused in multiple class hierarchies.
They are intended to provide member implementations en masse.
To use a mixin, use the `with` keyword followed by one or more mixin
names. The following example shows two classes that use (or, are subclasses of)
mixins:
To define a mixin, use the `mixin` declaration.
In the rare case where you need to define both a mixin _and_ a class, you can use
the `mixin class` declaration.
Mixins and mixin classes cannot have an `extends` clause,
and must not declare any generative constructors.
For example:

Contoh Kode / Perintah CLI:

```
class Musician extends Performer [!with Musical!] {
  // ***
}

class Maestro extends Person [!with Musical, Aggressive, Demented!] {
  Maestro(String maestroName) {
    name = maestroName;
    canConduct = true;
  }
}

// ----------------------------------------

mixin Musical {
  bool canPlayPiano = false;
  bool canCompose = false;
  bool canConduct = false;

  void entertainMe() {
    if (canPlayPiano) {
      print('Playing piano');
    } else if (canConduct) {
      print('Waving hands');
    } else {
      print('Humming to self');
    }
  }
}
```

[dart_060] Dart - Modern OOP - Mixins

Mixins (Code reuse across multiple class hierarchies with on & mixin class): Specify members a mixin can call on itself

Sumber: ht

Specify members a mixin can call on itself
Sometimes a mixin depends on being able to invoke a method or access fields,
but can't define those members itself (because mixins can't use constructor
parameters to instantiate their own fields).
The following sections cover different strategies for ensuring any subclass
of a mixin defines any members the mixin's behavior depends on.

[dart_061] Dart - Modern OOP - Mixins

Mixins (Code reuse across multiple class hierarchies with on & mixin class): Define abstract members in the mixin

Sumber: ht

Define abstract members in the mixin
Declaring an abstract method in a mixin forces any type that uses
the mixin to define the abstract method upon which its behavior depends.
Access state in the mixin's subclass
Declaring abstract members also allows you to access state on the subclass
of a mixin, by calling getters which are defined as abstract on the mixin:

Contoh Kode / Perintah CLI:

```
mixin Musician {
  void playInstrument(String instrumentName); // Abstract method.
```

```
  void playPiano() {
    playInstrument('Piano');
  }
  void playFlute() {
    playInstrument('Flute');
  }
}

class Virtuoso with Musician {

  @override
  void playInstrument(String instrumentName) { // Subclass must define.
    print('Plays the $instrumentName beautifully');
  }
}

// ----------------------------------------

/// Can be applied to any type with a [name] property and provides an
/// implementation of [hashCode] and operator `==` in terms of it.
mixin NameIdentity {
  String get name;

  @override
  int get hashCode => name.hashCode;

  @override
  bool operator ==(other) => other is NameIdentity && name == other.name;
}

class Person with NameIdentity {
  final String name;

  Person(this.name);
}
```

[dart_062] Dart - Modern OOP - Mixins

Mixins (Code reuse across multiple class hierarchies with on & mixin class): Implement an interface

Sumber: ht

Implement an interface

Similar to declaring the mixin abstract, putting an `implements` clause on the mixin while not actually implementing the interface will also ensure any member dependencies are defined for the mixin.

Contoh Kode / Perintah CLI:

```
abstract interface class Tuner {
  void tuneInstrument();
}

mixin Guitarist implements Tuner {
  void playSong() {
    tuneInstrument();

    print('Strums guitar majestically.');
  }
}

class PunkRocker with Guitarist {
```

```
  @override
  void tuneInstrument() {
    print("Don't bother, being out of tune is punk rock.");
  }
}
```

[dart_063] Dart - Modern OOP - Mixins

Mixins (Code reuse across multiple class hierarchies with on & mixin class): Use the `on` clause to declare a superclass

Sumber: ht

Use the `on` clause to declare a superclass
The `on` clause exists to define the type that `super` calls are resolved against.
So, you should only use it if you need to have a `super` call inside a mixin.
The `on` clause forces any class that uses a mixin to also be a subclass
of the type in the `on` clause.
If the mixin depends on members in the superclass,
this ensures those members are available where the mixin is used:
In this example, only classes that extend the `Musician` class
A class can mix in `MusicalPerformer` only if
its superclass is a subtype of `Musician` that
provides a concrete implementation of `musicianMethod()`.
Because `SingerDancer` extends `Musician`,
it can mix in `MusicalPerformer`.

Contoh Kode / Perintah CLI:

```
class Musician {
  musicianMethod() {
    print('Playing music!');
  }
}

mixin MusicalPerformer [!on Musician!] {
  performerMethod() {
    print('Performing music!');
    super.musicianMethod();
  }
}

class SingerDancer extends Musician with MusicalPerformer { }

main() {
  SingerDancer().performerMethod();
}
```

[dart_064] Dart - Modern OOP - Mixins

Mixins (Code reuse across multiple class hierarchies with on & mixin class): `class`, `mixin`, or `mixin class`?

Sumber: ht

`class`, `mixin`, or `mixin class`?
:::version-note
The `mixin class` declaration requires a [language version][] of at least 3.0.
:::
A `mixin` declaration defines a mixin. A `class` declaration defines a [class][].

A `mixin class` declaration defines a class that is usable as both a regular class
and a mixin, with the same name and the same type.
Any restrictions that apply to classes or mixins also apply to mixin classes:
- Mixins can't have `extends` or `with` clauses, so neither can a `mixin class`.
- Classes can't have an `on` clause, so neither can a `mixin class`.
[language version]: /language/versioning
[class]: /language/classes
[class modifiers]: /language/class-modifiers

Contoh Kode / Perintah CLI:

```
mixin class Musician {
  // ...
}

class Novice with Musician { // Use Musician as a mixin
  // ...
}

class Novice extends Musician { // Use Musician as a class
  // ...
}
```

[dart_065] Dart - Asynchronous Programming (Future & Stream)

Asynchronous Support (async, await, Future<T>, Stream<T>, Error handling): Overview

Sumber: ht

Dart libraries are full of functions that
return [`Future`][] or [`Stream`][] objects.
These functions are _asynchronous_:
they return after setting up
a possibly time-consuming operation
(such as I/O),
without waiting for that operation to complete.
The `async` and `await` keywords support asynchronous programming,
letting you write asynchronous code that
looks similar to synchronous code.

[dart_066] Dart - Asynchronous Programming (Future & Stream)

Asynchronous Support (async, await, Future<T>, Stream<T>, Error handling): Handling Futures

Sumber: ht

Handling Futures
When you need the result of a completed Future,
you have two options:
* Use `async` and `await`, as described here and in the
asynchronous programming tutorial.
* Use the Future API, as described in the
`dart:async` documentation.
Code that uses `async` and `await` is asynchronous,
but it looks a lot like synchronous code.
For example, here's some code that uses `await`
to wait for the result of an asynchronous function:
To use `await`, code must be in an `async` function -- a
function marked as `async`:

:::note
Although an `async` function might perform time-consuming operations,
it doesn't wait for those operations.
Instead, the `async` function executes only
until it encounters its first `await` expression.
Then it returns a `Future` object,
resuming execution only after the `await` expression completes.
:::
Use `try`, `catch`, and `finally` to handle errors and cleanup
in code that uses `await`:
You can use `await` multiple times in an `async` function.
For example, the following code waits three times
for the results of functions:
In await expression,
the value of expression is usually a Future;
if it isn't, then the value is automatically wrapped in a Future.
This Future object indicates a promise to return an object.
The value of await expression is that returned object.
The await expression makes execution pause until that object is available.
**If you get a compile-time error when using `await`,
make sure `await` is in an `async` function.**
For example, to use `await` in your app's `main()` function,
the body of `main()` must be marked as `async`:
:::note
The preceding example uses an `async` function (`checkVersion()`)
without waiting for a result -- a practice that can cause problems
if the code assumes that the function has finished executing.

Contoh Kode / Perintah CLI:

```
await lookUpVersion();

// ----------------------------------------

Future<void> checkVersion() [!async!] {
  var version = [!await!] lookUpVersion();
  // Do something with version
}

// ----------------------------------------

try {
  version = await lookUpVersion();
} catch (e) {
  // React to inability to look up the version
}
```

[dart_067] Dart - Asynchronous Programming (Future & Stream)

Asynchronous Support (async, await, Future<T>, Stream<T>, Error handling): Declaring async functions

Sumber: ht

Declaring async functions
An `async` function is a function whose body is marked with
the `async` modifier.
Adding the `async` keyword to a function makes it return a Future.

For example, consider this synchronous function,
which returns a String:
If you change it to be an `async` function -- for example,
because a future implementation will be time consuming -- the
returned value is a Future:
Note that the function's body doesn't need to use the Future API.
Dart creates the Future object if necessary.
If your function doesn't return a useful value,
make its return type `Future`.
For an interactive introduction to using futures, `async`, and `await`,
see the asynchronous programming tutorial.
TODO #1117: Where else should we cover generalized void?

Contoh Kode / Perintah CLI:

```
String lookUpVersion() => '1.0.0';

// ----------------------------------------

Future<String> lookUpVersion() async => '1.0.0';
```

[dart_068] Dart - Asynchronous Programming (Future & Stream)

Asynchronous Support (async, await, Future<T>, Stream<T>, Error handling): Handling Streams

Sumber: ht

Handling Streams
When you need to get values from a Stream,
you have two options:
* Use `async` and an _asynchronous for loop_ (`await for`).
* Use the Stream API, as described in the
`dart:async` documentation.
:::note
Before using `await for`, be sure that it makes the code clearer and that you
really do want to wait for all of the stream's results. For example, you
usually should **not** use `await for` for UI event listeners, because UI
frameworks send endless streams of events.
:::
An asynchronous for loop has the following form:
The value of expression must have type Stream.
Execution proceeds as follows:
1. Wait until the stream emits a value.
2. Execute the body of the for loop,
with the variable set to that emitted value.
3. Repeat 1 and 2 until the stream is closed.
To stop listening to the stream,
you can use a `break` or `return` statement,
which breaks out of the for loop
and unsubscribes from the stream.
**If you get a compile-time error when implementing an asynchronous for loop,
make sure the `await for` is in an `async` function.**
For example, to use an asynchronous for loop in your app's `main()` function,
the body of `main()` must be marked as `async`:
For more information about Dart's asynchronous programming support,

check out the `dart:async` library documentation.
[`Future`]: {{site.dart-api}}/dart-async/Future-class.html
[`Stream`]: {{site.dart-api}}/dart-async/Stream-class.html
[unawaited_futures linter rule]: /tools/linter-rules/unawaited_futures

Contoh Kode / Perintah CLI:

```
await for (varOrType identifier in expression) {
  // Executes each time the stream emits a value.
}

// ----------------------------------------

void main() [!async!] {
  // ...
  [!await for!] (final request in requestServer) {
    handleRequest(request);
  }
  // ...
}
```

[dart_069] Dart - Modern Features - Dart 3: Records

Dart 3 Records (Anonymous, immutable, aggregate values with positional & named fields): Overview

Sumber: ht

:::version-note
Records require a [language version][] of at least 3.0.
:::
Records are an anonymous, immutable, aggregate type. Like other [collection types][],
they let you bundle multiple objects into a single object. Unlike other collection
types, records are fixed-sized, heterogeneous, and typed.
Records are real values; you can store them in variables,
nest them, pass them to and from functions,
and store them in data structures such as lists, maps, and sets.

[dart_070] Dart - Modern Features - Dart 3: Records

Dart 3 Records (Anonymous, immutable, aggregate values with positional & named fields): Record syntax

Sumber: ht

Record syntax
Records expressions are comma-delimited lists of named or positional fields,
enclosed in parentheses:
Record type annotations are comma-delimited lists of types enclosed in parentheses.
You can use record type annotations to define return types and parameter types.
For example, the following `(int, int)` statements are record type annotations:
Fields in record expressions and type annotations mirror
how [parameters and arguments][] work in functions.
Positional fields go directly inside the parentheses:
In a record type annotation, named fields go inside a curly brace-delimited
section of type-and-name pairs, after all positional fields. In a record
expression, the names go before each field value with a colon after:
The names of named fields in a record type are part of
the record's type definition, or its _shape_.
Two records with named fields with

different names have different types:
In a record type annotation, you can also name the *positional* fields, but
these names are purely for documentation and don't affect the record's type:
This is similar to how positional parameters
in a [function declaration or function typedef][function-type] can have names,
but those names don't affect the signature of the function.
For more information and examples, check out
Record types and Record equality.

Contoh Kode / Perintah CLI:

```
var record = ('first', a: 2, b: true, 'last');

// ----------------------------------------

(int, int) swap((int, int) record) {
  var (a, b) = record;
  return (b, a);
}

// ----------------------------------------

// Record type annotation in a variable declaration:
(String, int) record;

// Initialize it with a record expression:
record = ('A string', 123);
```

[dart_071] Dart - Modern Features - Dart 3: Records

Dart 3 Records (Anonymous, immutable, aggregate values with positional & named fields): Record fields

Sumber: ht

Record fields
Record fields are accessible through built-in getters. Records are immutable,
so fields do not have setters.
Named fields expose getters of the same name. Positional fields expose getters
of the name `$`, skipping named fields:
To streamline record field access even more,
check out the page on [Patterns][pattern].

Contoh Kode / Perintah CLI:

```
var record = ('first', a: 2, b: true, 'last');

print(record.$1); // Prints 'first'
print(record.a); // Prints 2
print(record.b); // Prints true
print(record.$2); // Prints 'last'
```

[dart_072] Dart - Modern Features - Dart 3: Records

Dart 3 Records (Anonymous, immutable, aggregate values with positional & named fields): Record types

Sumber: ht

Record types
There is no type declaration for individual record types.
Records are structurally typed based on the types of their fields.
A record's _shape_ (the set of its fields, the fields' types,

and their names, if any) uniquely determines the type of a record.
Each field in a record has its own type. Field types can differ within the same
record. The type system is aware of each field's type wherever it is accessed
from the record:
Consider two unrelated libraries that create records with the same set of fields.
The type system understands that those records are the same type even though the
libraries are not coupled to each other.
:::tip
While you can't declare a unique type for a record shape,
you can create type aliases for readability and reuse.
To learn how and when to do so,
check out Records and typedefs.
:::

Contoh Kode / Perintah CLI:

```
(num, Object) pair = (42, 'a');

var first = pair.$1; // Static type `num`, runtime type `int`.
var second = pair.$2; // Static type `Object`, runtime type `String`.
```

[dart_073] Dart - Modern Features - Dart 3: Records

Dart 3 Records (Anonymous, immutable, aggregate values with positional & named fields): Record equality

Sumber: ht

Record equality
Two records are equal if they have the same _shape_ (set of fields),
and their corresponding fields have the same values.
Since named field _order_ is not part of a record's shape, the order of named
fields does not affect equality.
For example:
Records automatically define `hashCode` and `==` methods based on the structure
of their fields.

Contoh Kode / Perintah CLI:

```
(int x, int y, int z) point = (1, 2, 3);
(int r, int g, int b) color = (1, 2, 3);

print(point == color); // Prints 'true'.

// ----------------------------------------

({int x, int y, int z}) point = (x: 1, y: 2, z: 3);
({int r, int g, int b}) color = (r: 1, g: 2, b: 3);

print(point == color); // Prints 'false'. Lint: Equals on unrelated types.
```

[dart_074] Dart - Modern Features - Dart 3: Records

Dart 3 Records (Anonymous, immutable, aggregate values with positional & named fields): Multiple returns

Sumber: ht

Multiple returns
Records allow functions to return multiple values bundled together.
To retrieve record values from a return,
[destructure][] the values into local variables using [pattern matching][pattern].

You can also destructure a record using its named fields,
using the colon `:` syntax, which you can read more about on the
[Pattern types][] page:
You can return multiple values from a function without records,
but other methods come with downsides.
For example, creating a class is much more verbose, and using other collection
types like `List` or `Map` loses type safety.
:::note
Records' multiple-return and heterogeneous-type characteristics enable
parallelization of futures of different types, which you can read about in the
[`dart:async` documentation][].
:::

Contoh Kode / Perintah CLI:

```
// Returns multiple values in a record:
(String name, int age) userInfo(Map<String, dynamic> json) {
  return (json['name'] as String, json['age'] as int);
}

final json = <String, dynamic>{'name': 'Dash', 'age': 10, 'color': 'blue'};

// Destructures using a record pattern with positional fields:
var (name, age) = userInfo(json);

/* Equivalent to:
  var info = userInfo(json);
  var name = info.$1;
  var age  = info.$2;
*/

// ----------------------------------------

({String name, int age}) userInfo(Map<String, dynamic> json)
// ***
// Destructures using a record pattern with named fields:
final (:name, :age) = userInfo(json);
```

[dart_075] Dart - Modern Features - Dart 3: Records

Dart 3 Records (Anonymous, immutable, aggregate values with positional & named fields): Records as simple data structures

Sumber: ht

Records as simple data structures
Records only hold data. When that's all you need,
they're immediately available and easy to use
without needing to declare any new classes.
For a simple list of data tuples that all have the same shape,
a *list of records* is the most direct representation.
Take this list of "button definitions", for example:
This code can be written directly without needing any additional declarations.

Contoh Kode / Perintah CLI:

```
final buttons = [
  (
    label: "Button I",
    icon: const Icon(Icons.upload_file),
    onPressed: () => print("Action -> Button I"),
  ),
```

```
  (
    label: "Button II",
    icon: const Icon(Icons.info),
    onPressed: () => print("Action -> Button II"),
  )
];
```

[dart_076] Dart - Modern Features - Dart 3: Records

Dart 3 Records (Anonymous, immutable, aggregate values with positional & named fields): Records and typedefs

Sumber: ht

Records and typedefs
You can choose to use [typedefs][] to give the record type itself a name,
and use that rather than writing out the full record type.
This method allows you to state that some fields can be null (`?`),
even if none of the current entries in the list have a null value.
Because record types are structural types, giving a name like `ButtonItem`
only introduces an alias that makes it easier to refer to the structural type:
`({String label, Icon icon, void Function()? onPressed})`.
Having all your code refer to a record type by its alias makes it easier to
later change the record's implementation without needing to update every reference.
Code can work with the given button definitions the same way it would
with simple class instances:
You could even decide to later change the record type to a class type to add methods:
Or to an [extension type][]:
And then create the list of button definitions using that type's constructors:
Again, all while not needing to change the code that uses that list.
Changing any type does require the code using it to be very careful about
not making assumptions. A type alias does not offer any protection or guarantee,
for the code using it as a reference, that the value being aliased is a record.
Extension types, also, offer little protection.
Only a class can provide full abstraction and encapsulation.
[language version]: /language/versioning
[collection types]: /language/collections
[pattern]: /language/patterns#destructuring-multiple-returns
[`dart:async` documentation]: /libraries/dart-async#handling-errors-for-multiple-futures
[parameters and arguments]: /language/functions#parameters
[function-type]: /language/functions#function-types
[destructure]: /language/patterns#destructuring
[Pattern types]: /language/pattern-t

Contoh Kode / Perintah CLI:

```
typedef ButtonItem = ({String label, Icon icon, void Function()? onPressed});
final List<ButtonItem> buttons = [
  // ...
];

// ----------------------------------------

List<Container> widget = [
  for (var button in buttons)
    Container(
      margin: const EdgeInsets.all(4.0),
      child: OutlinedButton.icon(
        onPressed: button.onPressed,
```

```
        icon: button.icon,
        label: Text(button.label),
      ),
    ),
];

// ----------------------------------------

class ButtonItem {
  final String label;
  final Icon icon;
  final void Function()? onPressed;
  ButtonItem({required this.label, required this.icon, this.onPressed});
  bool get hasOnPressed => onPressed != null;
}
```

[dart_077] Dart - Modern Features - Dart 3: Patterns & Pattern Matching

Dart 3 Patterns (Destructuring, Pattern matching in switch/if-case, Exhaustiveness): Overview

Sumber: ht

:::version-note
Patterns require a [language version][] of at least 3.0.
:::
Patterns are a syntactic category in the Dart language,
like statements and expressions.
A pattern represents the shape of a set of values that
it might match against actual values.
This page describes:
- What patterns do.
- Where patterns are allowed in Dart code.
- What the common use cases for patterns are.
To learn about the different kinds of patterns,
visit the [pattern types][types] page.

[dart_078] Dart - Modern Features - Dart 3: Patterns & Pattern Matching

Dart 3 Patterns (Destructuring, Pattern matching in switch/if-case, Exhaustiveness): What patterns do

Sumber: ht

What patterns do
In general, a pattern can **match** a value, **destructure** a value, or both,
depending on the context and shape of the pattern.
First, _pattern matching_ allows you to check whether a given value:
- Has a certain shape.
- Is a certain constant.
- Is equal to something else.
- Has a certain type.
Then, _pattern destructuring_ provides you with a convenient declarative syntax to
break that value into its constituent parts. The same pattern can also let you
bind variables to some or all of those parts in the process.

[dart_079] Dart - Modern Features - Dart 3: Patterns & Pattern Matching

Dart 3 Patterns (Destructuring, Pattern matching in switch/if-case, Exhaustiveness): Matching

Sumber: ht

Matching
A pattern always tests against a value to determine if the value has the form
you expect. In other words, you are checking if the value _matches_ the pattern.
What constitutes a match depends on [what kind of pattern][types] you are using.
For example, a constant pattern matches if the value is equal to the pattern's
constant:
Many patterns make use of subpatterns, sometimes called _outer_ and _inner_
patterns, respectively. Patterns match recursively on their subpatterns.
For example, the individual fields of any [collection-type][] pattern could be
[variable patterns][variable] or [constant patterns][constant]:
To ignore parts of a matched value, you can use a [wildcard pattern][]
as a placeholder. In the case of list patterns, you can use a [rest element][].

Contoh Kode / Perintah CLI:

```
switch (number) {
  // Constant pattern matches if 1 == number.
  case 1:
    print('one');
}

// ----------------------------------------

const a = 'a';
const b = 'b';
switch (obj) {
  // List pattern [a, b] matches obj first if obj is a list with two fields,
  // then if its fields match the constant subpatterns 'a' and 'b'.
  case [a, b]:
    print('$a, $b');
}
```

[dart_080] Dart - Modern Features - Dart 3: Patterns & Pattern Matching

Dart 3 Patterns (Destructuring, Pattern matching in switch/if-case, Exhaustiveness): Destructuring

Sumber: ht

Destructuring
When an object and pattern match, the pattern can then access the object's data
and extract it in parts. In other words, the pattern _destructures_ the object:
You can nest [any kind of pattern][types] inside a destructuring pattern.
For example, this case pattern matches and destructures a two-element
list whose first element is `'a'` or `'b'`:

Contoh Kode / Perintah CLI:

```
var numList = [1, 2, 3];
// List pattern [a, b, c] destructures the three elements from numList...
var [a, b, c] = numList;
// ...and assigns them to new variables.
print(a + b + c);

// ----------------------------------------

switch (list) {
```

```
  case ['a' || 'b', var c]:
    print(c);
}
```

[dart_081] Dart - Modern Features - Dart 3: Patterns & Pattern Matching

Dart 3 Patterns (Destructuring, Pattern matching in switch/if-case, Exhaustiveness): Places patterns can appear

Sumber: ht

Places patterns can appear
You can use patterns in several places in the Dart language:
- Local variable declarations and assignments
- [for and for-in loops][for]
- [if-case][if] and [switch-case][switch]
- Control flow in [collection literals][]
This section describes common use cases for matching and destructuring with patterns.

[dart_082] Dart - Modern Features - Dart 3: Patterns & Pattern Matching

Dart 3 Patterns (Destructuring, Pattern matching in switch/if-case, Exhaustiveness): Variable declaration

Sumber: ht

Variable declaration
You can use a _pattern variable declaration_ anywhere Dart allows local variable declaration.
The pattern matches against the value on the right of the declaration.
Once matched, it destructures the value and binds it to new local variables:
A pattern variable declaration must start with either `var` or `final`, followed by a pattern.

Contoh Kode / Perintah CLI:

```
// Declares new variables a, b, and c.
var (a, [b, c]) = ('str', [1, 2]);
```

[dart_083] Dart - Modern Features - Dart 3: Patterns & Pattern Matching

Dart 3 Patterns (Destructuring, Pattern matching in switch/if-case, Exhaustiveness): Variable assignment

Sumber: ht

Variable assignment
A _variable assignment pattern_ falls on the left side of an assignment.
First, it destructures the matched object. Then it assigns the values to _existing_ variables, instead of binding new ones.
Use a variable assignment pattern to swap the values of two variables without declaring a third temporary one:

Contoh Kode / Perintah CLI:

```
var (a, b) = ('left', 'right');
(b, a) = (a, b); // Swap.
print('$a $b'); // Prints "right left".
```

[dart_084] Dart - Modern Features - Dart 3: Patterns & Pattern Matching

Dart 3 Patterns (Destructuring, Pattern matching in switch/if-case, Exhaustiveness): Switch statements and expressions

Sumber: ht

Switch statements and expressions
Every case clause contains a pattern. This applies to [switch statements][switch]
and [expressions][], as well as [if-case statements][if].
You can use [any kind of pattern][types] in a case.
Case patterns are [refutable][].
They allow control flow to either:
- Match and destructure the object being switched on.
- Continue execution if the object doesn't match.
The values that a pattern destructures in a case become local variables.
Their scope is only within the body of that case.
[Logical-or patterns][logical-or] are useful for having multiple cases share a
body in switch expressions or statements:
Switch statements can have multiple cases share a body
[without using logical-or patterns][share], but they are
still uniquely useful for allowing multiple cases to share a [guard][]:
[Guard clauses][guard] evaluate an arbitrary condition as part of a case, without
exiting the switch if the condition is false
(like using an `if` statement in the case body would cause).

Contoh Kode / Perintah CLI:

```
switch (obj) {
  // Matches if 1 == obj.
  case 1:
    print('one');

  // Matches if the value of obj is between the
  // constant values of 'first' and 'last'.
  case >= first && <= last:
    print('in range');

  // Matches if obj is a record with two fields,
  // then assigns the fields to 'a' and 'b'.
  case (var a, var b):
    print('a = $a, b = $b');

  default:
}

// ----------------------------------------

var isPrimary = switch (color) {
  Color.red || Color.yellow || Color.blue => true,
  _ => false,
};

// ----------------------------------------

switch (shape) {
  case Square(size: var s) || Circle(size: var s) when s > 0:
    print('Non-empty symmetric shape');
}
```

[dart_085] Dart - Modern Features - Dart 3: Patterns & Pattern Matching

Dart 3 Patterns (Destructuring, Pattern matching in switch/if-case, Exhaustiveness): For and for-in loops

Sumber: ht

For and for-in loops
You can use patterns in [for and for-in loops][for] to iterate-over and destructure values in a collection.
This example uses [object destructuring][object] in a for-in loop to destructure the [`MapEntry`][] objects that a `.entries` call returns:
The object pattern checks that `hist.entries` has the named type `MapEntry`, and then recurses into the named field subpatterns `key` and `value`.
It calls the `key` getter and `value` getter on the `MapEntry` in each iteration, and binds the results to local variables `key` and `count`, respectively.
Binding the result of a getter call to a variable of the same name is a common use case, so object patterns can also infer the getter name from the [variable subpattern][variable]. This allows you to simplify the variable pattern from something redundant like `key: key` to just `:key`:

Contoh Kode / Perintah CLI:

```
Map<String, int> hist = {'a': 23, 'b': 100};

for (var MapEntry(key: key, value: count) in hist.entries) {
  print('$key occurred $count times');
}

// ----------------------------------------

for (var MapEntry(:key, value: count) in hist.entries) {
  print('$key occurred $count times');
}
```

[dart_086] Dart - Modern Features - Dart 3: Patterns & Pattern Matching

Dart 3 Patterns (Destructuring, Pattern matching in switch/if-case, Exhaustiveness): Use cases for patterns

Sumber: ht

Use cases for patterns
The previous section
describes _how_ patterns fit into other Dart code constructs.
You saw some interesting use cases as examples, like swapping
the values of two variables, or
destructuring key-value pairs
in a map. This section describes even more use cases, answering:
- _When and why_ you might want to use patterns.
- What kinds of problems they solve.
- Which idioms they best suit.

[dart_087] Dart - Modern Features - Dart 3: Patterns & Pattern Matching

Dart 3 Patterns (Destructuring, Pattern matching in switch/if-case, Exhaustiveness): Destructuring multiple returns

Sumber: ht

Destructuring multiple returns
Records allow aggregating and [returning multiple values][] from a single
function call. Patterns add the ability to destructure a record's fields
directly into local variables, inline with the function call.
Instead of individually declaring new local variables for each record field,
like this:
You can destructure the fields of a record that a function returns into local
variables using a variable declaration or
assignment pattern, and a [record pattern][record]
as its subpattern:
To destructure a record with named fields using a pattern:

Contoh Kode / Perintah CLI:

```
var info = userInfo(json);
var name = info.$1;
var age = info.$2;

// ----------------------------------------

var (name, age) = userInfo(json);

// ----------------------------------------

final (:name, :age) =
    getData(); // For example, return (name: 'doug', age: 25);
```

[dart_088] Dart - Modern Features - Dart 3: Patterns & Pattern Matching

Dart 3 Patterns (Destructuring, Pattern matching in switch/if-case, Exhaustiveness): Destructuring class instances

Sumber: ht

Destructuring class instances
[Object patterns][object] match against named object types, allowing
you to destructure their data using the getters the object's class already exposes.
To destructure an instance of a class, use the named type,
followed by the properties to
destructure enclosed in parentheses:

Contoh Kode / Perintah CLI:

```
final Foo myFoo = Foo(one: 'one', two: 2);
var Foo(:one, :two) = myFoo;
print('one $one, two $two');
```

[dart_089] Dart - Modern Features - Dart 3: Patterns & Pattern Matching

Dart 3 Patterns (Destructuring, Pattern matching in switch/if-case, Exhaustiveness): Algebraic data types

Sumber: ht

Algebraic data types
Object destructuring and switch cases are conducive to writing
code in an [algebraic data type][] style.
Use this method when:
- You have a family of related types.
- You have an operation that needs specific behavior for each type.
- You want to group that behavior in one place instead of spreading it across all
the different type definitions.
Instead of implementing the operation as an instance method for every type,
keep the operation's variations in a single function that switches over the subtypes:

Contoh Kode / Perintah CLI:

```
sealed class Shape {}

class Square implements Shape {
  final double length;
  Square(this.length);
}

class Circle implements Shape {
  final double radius;
  Circle(this.radius);
}

double calculateArea(Shape shape) => switch (shape) {
  Square(length: var l) => l * l,
  Circle(radius: var r) => math.pi * r * r,
};
```

[dart_090] Dart - Modern Features - Dart 3: Patterns & Pattern Matching

Dart 3 Patterns (Destructuring, Pattern matching in switch/if-case, Exhaustiveness): Validating incoming JSON

Sumber: ht

Validating incoming JSON
[Map][] and [list][] patterns work well for destructuring key-value pairs in
deserialized data, such as data parsed from JSON:
If you know that the JSON data has the structure you expect,
the previous example is realistic.
But data typically comes from an external source, like over the network.
You need to validate it first to confirm its structure.
Without patterns, validation is verbose:
A single case pattern
can achieve the same validation.
Single cases work best as [if-case][if] statements.
Patterns provide a more declarative, and much less verbose
method of validating JSON:
This case pattern simultaneously validates that:
- `data` is a map, because it must first match the outer
[map pattern][map] to proceed.

- And, since it's a map, it also confirms `data` is not null.
- `data` contains a key `user`.
- The key `user` pairs with a list of two values.
- The types of the list values are `String` and `int`.
- The new local variables to hold the values are `name` and `age`.

[language version]: /language/versioning
[types]: /language/pattern-types
[collection-type]: /language/collections
[wildcard pattern]: /language/pattern-types#wildcard
[rest element]: /language/pattern-types#rest-element
[null-check pattern]: /language/pattern-types#null-check
[for]: /language/loops#for-loops
[if]: /language/branches#if-case
[switch]: /language/branches#switch-statements
[expressions]: /language/branches#switch-expressions
[collection literals]: /language/collections#control-flow-operators
[null-assert pattern]: /language/pattern-types#null-assert
[record]: /language/pattern-types#record
[returning multiple values]: /language/records#multiple-returns
[refutable]: /resources/glossary#refutable-pattern
[constant]: /language/pattern-types#constant
[list]: /l

Contoh Kode / Perintah CLI:

```
var data = {
  'user': ['Lily', 13],
};
var {'user': [name, age]} = data;

// ----------------------------------------

if (data is Map<String, Object?> && data.containsKey('user')) {
  var user = data['user'];
  if (user is List<Object?> &&
      user.length == 2 &&
      user[0] is String &&
      user[1] is int) {
    var name = user[0] as String;
    var age = user[1] as int;
    print('User $name is $age years old.');
  }
}

// ----------------------------------------

if (data case {'user': [String name, int age]}) {
  print('User $name is $age years old.');
}
```

[dart_091] Dart - Modern Features - Dart 3: Class Modifiers

Dart 3 Class Modifiers (sealed, final, base, interface, abstract, mixin class): Overview

Sumber: ht

:::version-note
Class modifiers, besides `abstract`, require
a [language version][] of at least 3.0.
:::
Class modifiers control how a class or mixin can be used, both
from within its own library, and
from outside the library where it's defined.
Modifier keywords come before a class or mixin declaration.
For example, writing `abstract class` defines an abstract class.
The full set of modifiers that can appear before a class declaration include:
- `abstract`
- `base`
- `final`
- `interface`
- `sealed`
- [`mixin`][class, mixin, or mixin class]

Only the `base` modifier can appear before a mixin declaration.
The modifiers do not apply to other declarations like
`enum`, `typedef`, `extension`, or `extension type`.
When deciding whether to use class modifiers,
consider the intended uses of the class and
what behaviors the class needs to be able to rely on.
:::tip
If you're already familiar with Dart's class modifiers and just want
an outline or refresher of their behavior when combined,
check out the [Class modifier reference][].
If you maintain a library,
read the [Class modifiers for API maintainers][] page for
guidance on how to navigate these changes for your libraries.
:::
[Class modifier reference]: /language/modifier-reference
[Class modifiers for API maintainers]: /language/class-modifiers-for-apis

[dart_092] Dart - Modern Features - Dart 3: Class Modifiers

Dart 3 Class Modifiers (sealed, final, base, interface, abstract, mixin class): No modifier

Sumber: ht

No modifier
To allow unrestricted permission to construct or subtype from any library,
use a `class` or `mixin` declaration without a modifier. By default, you can:
- [Construct][] new instances of a class.
- [Extend][] a class to create a new subtype.
- [Implement][] a class or mixin's interface.
- [Mix in][mixin] a mixin or mixin class.

[dart_093] Dart - Modern Features - Dart 3: Class Modifiers

Dart 3 Class Modifiers (sealed, final, base, interface, abstract, mixin class): `abstract`

Sumber: ht

`abstract`
To define a class that doesn't require a full, concrete implementation of
its entire interface, use the `abstract` modifier.
Abstract classes cannot be constructed from any library, whether its own or
an outside library. Abstract classes often have [abstract methods][].
If you want your abstract class to appear to be instantiable,
define a [factory constructor][].

Contoh Kode / Perintah CLI:

```
<?code-excerpt "language/lib/class_modifiers/ex1/b.dart (abstract-usages)"?>
```

[dart_094] Dart - Modern Features - Dart 3: Class Modifiers

Dart 3 Class Modifiers (sealed, final, base, interface, abstract, mixin class): `base`

Sumber: ht

`base`
To enforce inheritance of a class or mixin's implementation,
use the `base` modifier.
A base class disallows implementation outside of its own library.
This guarantees:
- The base class constructor is called whenever
an instance of a subtype of the class is created.
- All implemented private members exist in subtypes.
- A new implemented member in a `base` class does not break subtypes,
since all subtypes inherit the new member.
- This is true unless the subtype already declares a member with
the same name and an incompatible signature.
You must mark any class that implements or extends a base class as
`base`, `final`, or `sealed`. This prevents outside libraries from
breaking the base class guarantees.

Contoh Kode / Perintah CLI:

```
<?code-excerpt "language/lib/class_modifiers/ex2/b.dart"?>
```

[dart_095] Dart - Modern Features - Dart 3: Class Modifiers

Dart 3 Class Modifiers (sealed, final, base, interface, abstract, mixin class): `interface`

Sumber: ht

`interface`
To define an interface, use the `interface` modifier.
Libraries outside of the interface's own defining library can
implement the interface, but not extend it.
This guarantees:
- When one of the class's instance methods calls
another instance method on `this`, it will always
invoke a known implementation of the method from the same library.
- Other libraries can't override methods that the interface
class's own methods might later call in unexpected ways.
This reduces the [fragile base class problem][].

Contoh Kode / Perintah CLI:

```
<?code-excerpt "language/lib/class_modifiers/ex3/b.dart"?>
```

[dart_096] Dart - Modern Features - Dart 3: Class Modifiers

Dart 3 Class Modifiers (sealed, final, base, interface, abstract, mixin class): `abstract interface`

Sumber: ht

`abstract interface`
The most common use for the `interface` modifier is to define a pure interface.
Combine the `interface` and `abstract`
modifiers for an `abstract interface class`.
Like an `interface` class, other libraries can
implement, but can't inherit, a pure interface.
Like an `abstract` class, a pure interface can have abstract members.

Contoh Kode / Perintah CLI:

```
<?code-excerpt "language/lib/class_modifiers/ex6/b.dart"?>
```

[dart_097] Dart - Modern Features - Dart 3: Class Modifiers

Dart 3 Class Modifiers (sealed, final, base, interface, abstract, mixin class): `final`

Sumber: ht

`final`
To close the type hierarchy, use the `final` modifier.
This prevents subtyping from a class outside of the current library.
Disallowing both inheritance and implementation prevents subtyping entirely.
This guarantees:
- You can safely add incremental changes to the API.
- You can call instance methods knowing that they haven't been overwritten in
a third-party subclass.
Final classes can be extended or implemented within the same library.
The `final` modifier encompasses the effects of `base`, and
therefore any subclasses must also be marked `base`, `final`, or `sealed`.

Contoh Kode / Perintah CLI:

```
<?code-excerpt "language/lib/class_modifiers/ex4/b.dart"?>
```

[dart_098] Dart - Modern Features - Dart 3: Class Modifiers

Dart 3 Class Modifiers (sealed, final, base, interface, abstract, mixin class): `sealed`

Sumber: ht

`sealed`
To create a known, enumerable set of subtypes, use the `sealed` modifier.
This allows you to create a switch over those subtypes that
is statically ensured to be [_exhaustive_][exhaustive].
The `sealed` modifier prevents a class from being extended or
implemented outside its own library. Sealed classes are implicitly
abstract.
- They cannot be constructed themselves.
- They can have factory constructors.
- They can define constructors for their subclasses to use.
Subclasses of sealed classes are, however, not implicitly abstract.
The compiler is aware of any possible direct subtypes
because they can only exist in the same library.
This allows the compiler to alert you when a switch does not
exhaustively handle all possible subtypes in its cases:

If you don't want [exhaustive switching][exhaustive],
or want to be able to add subtypes later without breaking the API,
use the `final` modifier. For a more in depth comparison,
read `sealed` versus `final`.

Contoh Kode / Perintah CLI:

```
sealed class Vehicle {}

class Car extends Vehicle {}

class Truck implements Vehicle {}

class Bicycle extends Vehicle {}

// ERROR: `Vehicle` can't be instantiated because
// it is marked `sealed` and therefore, implicitly abstract.
Vehicle myVehicle = Vehicle();

// Subclasses of a sealed class can be instantiated unless also restricted.
Vehicle myCar = Car();

extension VehicleSounds on Vehicle {
  String get sound {
    // ERROR: The switch does not exhaustively account for
    // all possible objects of type `Vehicle`.
    // In this example, a `Vehicle` with a run-time type of `Bicycle`
    // would not match any of the cases.
    return switch (this) {
      Car() => 'vroom',
      Truck() => 'VROOOOMM',
    };
  }
}
```

[dart_099] Dart - Modern Features - Dart 3: Class Modifiers

Dart 3 Class Modifiers (sealed, final, base, interface, abstract, mixin class): Combining modifiers

Sumber: ht

Combining modifiers
You can combine some modifiers for layered restrictions.
A class declaration can be, in order:
1. (Optional) `abstract`, describing whether the class can
contain abstract members and prevents instantiation.
2. (Optional) One of `base`, `interface`, `final` or `sealed`, describing
restrictions on other libraries subtyping the class.
3. (Optional) `mixin`, describing whether the declaration can be mixed in.
4. The `class` keyword itself.
You can't combine some modifiers because they are
contradictory, redundant, or otherwise mutually exclusive:
* `abstract` with `sealed`. A sealed class is
implicitly abstract.
* `interface`, `final` or `sealed` with `mixin`. These access modifiers
prevent [mixing in][mixin].
For further guidance on how class modifiers can be combined,
check out the [Class modifiers reference][].
[Class modifiers reference]: /language/modifier-reference

[language version]: /language/versioning
[class, mixin, or mixin class]: /language/mixins#class-mixin-or-mixin-class
[mixin]: /language/mixins
[fragile base class problem]: https://en.wikipedia.org/wiki/Fragile_base_class
[`noSuchMethod`]: /language/extend#nosuchmethod
[construct]: /language/constructors
[extend]: /language/extend
[implement]: /language/classes#implicit-interfaces
[factory constructor]: /language/constructors#factory-constructors
[exhaustive]: /language/branches#exhaustiveness-checking
[abstract methods]: /language/methods#abstract-methods
[syntax specification]: {{site.repo.dart.lang}}/blob/main/accepted/3.0/class-modifiers/feature-specification.md#syntax

[dart_100] Dart - Modern Features - Dart 3.3: Extension Types

Dart 3.3 Extension Types (Zero-cost compile-time wrappers with representation type): Overview

Sumber: ht

An extension type is a compile-time abstraction that "wraps"
an existing type with a different, static-only interface.
They are a major component of [static JS interop][] because
they can easily modify an existing type's interface
(crucial for any kind of interop)
without incurring the cost of an actual wrapper.
Extension types enforce discipline on the set of operations (or interface)
available to objects of an underlying type,
called the *representation type*.
When defining the interface of an extension type,
you can choose to reuse some members of the representation type,
omit others, replace others, and add new functionality.
The following example wraps the `int` type to create an extension type
that only allows operations that make sense for ID numbers:
:::note
Extension types serve the same purpose as **wrapper classes**,
but don't require the creation of an extra run-time object,
which can get expensive when you need to wrap lots of objects.
Because extension types are static-only and compiled away at run time,
they are essentially zero cost.
[**Extension methods**][ext] (also known just as "extensions")
are a static abstraction similar to extension types.
However, an extension method adds functionality *directly*
to every instance of its underlying type.
Extension types are different;
an extension type's interface *only* applies to expressions
whose static type is that extension type.
They are distinct from the interface of their underlying type by default.
:::
[static JS interop]: /go/next-gen-js-interop
[ext]: /language/extension-methods

Contoh Kode / Perintah CLI:

```
extension type IdNumber(int id) {
  // Wraps the 'int' type's '<' operator:
```

```
  operator <(IdNumber other) => id < other.id;
  // Doesn't declare the '+' operator, for example,
  // because addition does not make sense for ID numbers.
}

void main() {
  // Without the discipline of an extension type,
  // 'int' exposes ID numbers to unsafe operations:
  int myUnsafeId = 42424242;
  myUnsafeId = myUnsafeId + 10; // This works, but shouldn't be allowed for IDs.

  var safeId = IdNumber(42424242);
  safeId + 10; // Compile-time error: No '+' operator.
  myUnsafeId = safeId; // Compile-time error: Wrong type.
  myUnsafeId = safeId as int; // OK: Run-time cast to representation type.
  safeId < IdNumber(42424241); // OK: Uses wrapped '<' operator.
}
```

[dart_101] Dart - Modern Features - Dart 3.3: Extension Types

Dart 3.3 Extension Types (Zero-cost compile-time wrappers with representation type): Declaration

Sumber: ht

Declaration
Define a new extension type with the `extension type` declaration and a name,
followed by the *representation type declaration* in parentheses:
The representation type declaration `(int i)` specifies that
the underlying type of extension type `E` is `int`,
and that the reference to the *representation object* is named `i`.
The declaration also introduces:
- An implicit getter for the representation object
with the representation type as the return type: `int get i`.
- An implicit constructor: `E(int i) : i = i`.
The representation getter gives access to the
representation object typed as the underlying type.
The getter is in scope in the extension type body,
and you can access it using its name like any other getter:
- Within the extension type body using `i` (or `this.i`).
- Outside with a property extraction using `e.i`
(where `e` has the extension type as its static type).
Extension type declarations can also include [type parameters][generics]
just like classes or extensions:
[generics]: /language/generics

Contoh Kode / Perintah CLI:

```
extension type E(int i) {
  // Define set of operations.
}

// ----------------------------------------

extension type E<T>(List<T> elements) {
  // ...
}
```

[dart_102] Dart - Modern Features - Dart 3.3: Extension Types

Dart 3.3 Extension Types (Zero-cost compile-time wrappers with representation type): Constructors

Sumber: ht

Constructors
You can optionally declare [constructors][] in an extension type's body.
The representation declaration itself is an implicit constructor,
so by default takes the place of an unnamed constructor for the extension type.
Any additional non-redirecting generative constructors must
initialize the representation object's instance variable
using `this.i` in its initializer list or formal parameters.
Or, you can name the representation declaration constructor,
in which case there is room for an unnamed constructor in the body:
You can also completely hide the constructor, instead of just defining a new one,
using the same private constructor syntax for classes, `_`. For example,
if you only want clients constructing `E` with a `String`, even though
the underlying type is `int`:
You can also declare forwarding generative constructors,
or [factory constructors][factory]
(which can also forward to constructors of sub-extension types).
[constructors]: /language/constructors
[factory]: /language/constructors#factory-constructors

Contoh Kode / Perintah CLI:

```
extension type E(int i) {
  E.n(this.i);
  E.m(int j, String foo) : i = j + foo.length;
}

void main() {
  E(4); // Implicit unnamed constructor.
  E.n(3); // Named constructor.
  E.m(5, "Hello!"); // Named constructor with additional parameters.
}

// ----------------------------------------

extension type const E._(int it) {
  E(): this._(42);
  E.otherName(this.it);
}

void main2() {
  E();
  const E._(2);
  E.otherName(3);
}

// ----------------------------------------

extension type E._(int i) {
  E.fromString(String foo) : i = int.parse(foo);
}
```

[dart_103] Dart - Modern Features - Dart 3.3: Extension Types

Dart 3.3 Extension Types (Zero-cost compile-time wrappers with representation type): Members

Sumber: ht

Members
Declare members in the body of an extension type to define its interface
the same way you would for class members.
Extension type members can be methods, getters, setters, or operators
(non-[`external`][] [instance variables][] and [abstract members][]
are not allowed):
Interface members of the representation type are not interface members
of the extension type by default.
To make a single member of the representation type available
on the extension type, you must write a declaration for it
in the extension type definition, like the `operator +` in `NumberE`.
You also can define new members unrelated to the representation type,
like the `i` getter and `isValid` method.
[`external`]: /language/functions#external
[instance variables]: /language/classes#instance-variables
[abstract members]: /language/methods#abstract-methods

Contoh Kode / Perintah CLI:

```
extension type NumberE(int value) {
  // Operator:
  NumberE operator +(NumberE other) =>
      NumberE(value + other.value);
  // Getter:
  NumberE get myNum => this;
  // Method:
  bool isValid() => !value.isNegative;
}
```

[dart_104] Dart - Modern Features - Dart 3.3: Extension Types

Dart 3.3 Extension Types (Zero-cost compile-time wrappers with representation type): Implements

Sumber: ht

Implements
You can optionally use the `implements` clause to:
- Introduce a subtype relationship on an extension type, AND
- Add the members of the representation object to the extension type interface.
The `implements` clause introduces an [applicability][]
relationship like the one between an [extension method][ext] and its `on` type.
Members that are applicable to the supertype are applicable to the
subtype as well, unless the subtype has a declaration with the same
member name.
An extension type can only implement:
- **Its representation type**.
This makes all members of the representation type implicitly available
to the extension type.
- **A supertype of its representation type**.
This makes the members of the supertype available,
while not necessarily all the members of representation type.

- **Another extension type** that is valid on the same representation type.
This allows you to reuse operations across multiple extension types
(similar to multiple inheritance).
Read the Usage section to learn more about the effect of `implements`
in different scenarios.
[applicability]:
{{site.repo.dart.lang}}/blob/main/accepted/2.7/static-extension-methods/feature-specification.md#examples
[ext]: /language/extension-methods
`@redeclare`
Declaring an extension type member that
shares a name with a member of a supertype is *not* an
override relationship like it is between classes, but rather a *redeclaration*.
An extension type member declaration *completely replaces* any
supertype member with the same name.
It's not possible to provide an
alternative implementation for the same function.
You can use the [`@redeclare`][] annotation from `package:meta` to
tell the compiler you are *knowingly* choosing to
use the same name as a supertype's member.
The analyzer will the

Contoh Kode / Perintah CLI:

```
extension type NumberI(int i)
    implements int{
    // 'NumberI' can invoke all members of 'int',
    // plus anything else it declares here.
  }

// ----------------------------------------

extension type Sequence<T>(List<T> _) implements Iterable<T> {
    // Better operations than List.
  }

  extension type Id(int _id) implements Object {
    // Makes the extension type non-nullable.
    static Id? tryParse(String source) => int.tryParse(source) as Id?;
  }

// ----------------------------------------

extension type const Opt<T>._(({T value})? _) {
    const factory Opt(T value) = Val<T>;
    const factory Opt.none() = Non<T>;
  }
  extension type const Val<T>._(({T value}) _) implements Opt<T> {
    const Val(T value) : this._((value: value));
    T get value => _.value;
  }
  extension type const Non<T>._(Null _) implements Opt<Never> {
    const Non() : this._(null);
  }
```

[dart_105] Dart - Modern Features - Dart 3.3: Extension Types

Dart 3.3 Extension Types (Zero-cost compile-time wrappers with representation type): Usage

Sumber: ht

Usage
To use an extension type, create an instance the same as you would with a class:
by calling a constructor:
Then, you can invoke members on the object as you would with a class object.
There are two equally valid, but substantially different core use cases
for extension types:
1. Providing an *extended* interface to an existing type.
2. Providing a *different* interface to an existing type.
:::note
In any case, the representation type of an extension type is never its subtype,
so a representation type can't be used interchangeably where
the extension type is needed.
:::

Contoh Kode / Perintah CLI:

```
extension type NumberE(int value) {
  NumberE operator +(NumberE other) =>
      NumberE(value + other.value);

  NumberE get next => NumberE(value + 1);
  bool isValid() => !value.isNegative;
}

void testE() {
  var num = NumberE(1);
}
```

[dart_106] Dart - Modern Features - Dart 3.3: Extension Types

Dart 3.3 Extension Types (Zero-cost compile-time wrappers with representation type): 1. Provide an *extended* interface to an existing type {: #provide-extended-interface}

Sumber: ht

1. Provide an *extended* interface to an existing type {: #provide-extended-interface}
When an extension type implements its representation type,
you can consider it "transparent", because it allows the extension type
to "see" the underlying type.
A transparent extension type can invoke all members of the
representation type (that aren't redeclared),
plus any auxiliary members it defines.
This creates a new, *extended* interface for an existing type.
The new interface is available to expressions
whose static type is the extension type.
This means you *can* invoke members of the representation type
(unlike a non-transparent
extension type), like so:
You can also have a "mostly-transparent" extension type
that adds new members and adapts others by redeclaring a given member name
from the supertype.
This would allow you to use stricter types on some parameters of a method,
or different default values, for example.

Another mostly-transparent extension type approach is to implement
a type that is a supertype of the representation type.
For example, if the representation type is private but its supertype
defines the part of the interface that matters for clients.

Contoh Kode / Perintah CLI:

```
extension type NumberT(int value)
  implements int {
  // Doesn't explicitly declare any members of 'int'.
  NumberT get i => this;
}

void main () {
  // All OK: Transparency allows invoking `int` members on the extension type:
  var v1 = NumberT(1); // v1 type: NumberT
  int v2 = NumberT(2); // v2 type: int
  var v3 = v1.i - v1;  // v3 type: int
  var v4 = v2 + v1; // v4 type: int
  var v5 = 2 + v1; // v5 type: int
  // Error: Extension type interface is not available to representation type
  v2.i;
}
```

[dart_107] Dart - Modern Features - Dart 3.3: Extension Types

Dart 3.3 Extension Types (Zero-cost compile-time wrappers with representation type): 2. Provide a *different* interface to an existing type {:#provide-different-interface}

Sumber: ht

2. Provide a *different* interface to an existing type {:#provide-different-interface}
An extension type that is not transparent
(that does not `implement` its representation type)
is statically treated as a completely new type,
distinct from its representation type.
You can't assign it to its representation type,
and it doesn't expose its representation type's members.
For example, take the `NumberE` extension type we declared under Usage:
You can use an extension type this way to *replace* the interface
of an existing type. This allows you to model an interface that is
makes sense for the constraints of your new type
(like the `IdNumber` example in the introduction), while also benefitting from
the performance and convenience of a simple pre-defined type, like `int`.
This use case is as close as you can get to the complete encapsulation
of a wrapper class (but is realistically only a
somewhat protected abstraction).

Contoh Kode / Perintah CLI:

```
void testE() {
  var num1 = NumberE(1);
  int num2 = NumberE(2); // Error: Can't assign 'NumberE' to 'int'.

  num1.isValid(); // OK: Extension member invocation.
  num1.isNegative(); // Error: 'NumberE' does not define 'int' member 'isNegative'.

  var sum1 = num1 + num1; // OK: 'NumberE' defines '+'.
  var diff1 = num1 - num1; // Error: 'NumberE' does not define 'int' member '-'.
  var diff2 = num1.value - 2; // OK: Can access representation object with reference.
  var sum2 = num1 + 2; // Error: Can't assign 'int' to parameter type 'NumberE'.
```

```
  List<NumberE> numbers = [
    NumberE(1),
    num1.next, // OK: 'next' getter returns type 'NumberE'.
    1, // Error: Can't assign 'int' element to list type 'NumberE'.
  ];
}
```

[dart_108] Dart - Modern Features - Dart 3.3: Extension Types

Dart 3.3 Extension Types (Zero-cost compile-time wrappers with representation type): Type considerations

Sumber: ht

Type considerations
Extension types are a compile-time wrapping construct.
At run time, there is absolutely no trace of the extension type.
Any type query or similar run-time operations work on the representation type.
This makes extension types an *unsafe* abstraction,
because you can always find out the representation type at run time
and access the underlying object.
Dynamic type tests (`e is T`), casts (`e as T`),
and other run-time type queries (like `switch (e) ...` or `if (e case ...)`)
all evaluate to the underlying representation object,
and type check against that object's runtime type.
That's true when the static type of `e` is an extension type,
and when testing against an extension type (`case MyExtensionType(): ... `).
Similarly, the static type of the matched value is that of the extension type
in this example:
When `i` gets the static type `NumberE` based on a cast or a pattern match, `i`
still refers to the same object, and `v` refers to the same object as `i`, and
the object itself remains unchanged. It is only the static type that changed
(and hence the methods that we can call on this object). In particular, the
change of type does not involve an invocation of a constructor. If you wish to
execute a constructor (for example, to perform some kind of validation) then it
is necessary to write an explicit constructor invocation (such as `NumberE(i)`).
It's important to be aware of this behavior when using extension types.
Always keep in mind that an extension type exists and matters at compile time,
but gets _erased_ during compilation.
For example, consider an expression `e` whose static type is the
extension type `E`. Assume that the representation type of `E` is `R`.
The run-time type of the value of `e` is then a subtype of `R`.
Even the type itse

Contoh Kode / Perintah CLI:

```
void main() {
  var n = NumberE(1);

  // The run-time type of 'n' is the representation type 'int'.
  if (n is int) print(n); // Prints 1.

  // Can use 'int' methods on 'n' at run time.
  if (n case int x) print(x.toRadixString(10)); // Prints 1.
  switch (n) {
    case int(:var isEven): print("$n (${isEven ? "even" : "odd"})"); // Prints 1 (odd).
  }
}
```

```
// ----------------------------------------

void main() {
  int i = 2;
  if (i is NumberE) print("It is"); // Prints 'It is'.
  if (i case NumberE v) print("value: ${v.value}"); // Prints 'value: 2'.
  switch (i) {
    case NumberE(:var value): print("value: $value"); // Prints 'value: 2'.
  }
}
```